CS 241 — Foundations of Sequential Programs

Vishesh Gupta

University of Waterloo

Contents

1 Introduction and Data Representation **1**
- 1.1 What this course is about . . . 1
- 1.2 Bits, bytes and words . . . 1
- 1.3 Number bases . . . 2
  - 1.3.1 Binary and hexadecimal . . . 2
  - 1.3.2 Decimal to binary . . . 2
- 1.4 Unsigned integers . . . 2
- 1.5 Signed integers . . . 2
  - 1.5.1 Sign-magnitude (and why we do not use it) . . . 2
  - 1.5.2 One's complement . . . 3
  - 1.5.3 Two's complement . . . 3
  - 1.5.4 Sign extension . . . 3
- 1.6 Characters: ASCII . . . 3
- 1.7 Bit operations and shifts . . . 4
- 1.8 Endianness and byte addressing . . . 4
- 1.9 The stored-program idea . . . 4
- 1.10 Summary . . . 4

2 MIPS Machine Language **6**
- 2.1 The machine model . . . 6
  - 2.1.1 Register conventions . . . 6
- 2.2 The instruction set . . . 7
- 2.3 Instruction encodings . . . 7
- 2.4 Loader conventions: `mips.twoints` and `mips.array` . . . 9
- 2.5 Memory-mapped I/O . . . 9
- 2.6 First machine-language programs . . . 9
- 2.7 Summary . . . 10

3 Assembly Language Programming **11**
- 3.1 From machine code to assembly . . . 11
- 3.2 Labels . . . 11
- 3.3 Translating control structures . . . 12
  - 3.3.1 If-then-else . . . 12
  - 3.3.2 Comparisons . . . 12
  - 3.3.3 Loops . . . 12
- 3.4 Arrays . . . 12
- 3.5 The stack . . . 13

3.6 Procedures . . . 13
3.6.1 Calling conventions . . . 14
3.7 Stack frames and recursion . . . 15
3.8 Input and output . . . 16
3.9 Summary . . . 16

4 Assemblers **17**
4.1 The job of an assembler . . . 17
4.2 Scanning the input . . . 17
4.3 The forward-reference problem . . . 17
4.4 Two-pass assembly . . . 18
4.4.1 Pass 1: analysis and the symbol table . . . 18
4.4.2 Pass 2: synthesis . . . 18
4.5 Error checking . . . 18
4.6 Assembler design notes . . . 19
4.7 Beyond the CS 241 subset . . . 19
4.8 Summary . . . 19

5 Linking, Loading and Relocation **21**
5.1 Loaders . . . 21
5.2 Relocation . . . 21
5.3 The MERL format . . . 22
5.4 A relocating loader . . . 22
5.5 Linking . . . 23
5.5.1 Linking algorithm . . . 23
5.6 Static vs. dynamic linking . . . 24
5.7 Summary . . . 24

6 Formal Languages and Regular Languages **25**
6.1 Why formal languages? . . . 25
6.2 Basic definitions . . . 25
6.3 Regular languages . . . 26
6.3.1 Regular expressions . . . 26
6.4 Deterministic finite automata . . . 26
6.5 Nondeterministic finite automata . . . 27
6.5.1 Simulating an NFA . . . 27
6.5.2 Subset construction (NFA $\rightarrow$ DFA) . . . 28
6.6 ε-NFAs . . . 28
6.7 Kleene's theorem . . . 28
6.8 Summary . . . 29

7 Scanning **30**
7.1 The scanning problem . . . 30
7.2 Building a token-recognising DFA . . . 30
7.3 Maximal munch . . . 31
7.4 Simplified maximal munch . . . 32
7.5 Whitespace, comments, and token separation . . . 32
7.6 Theory: why not just a regex for the program? . . . 32

7.7 Summary . . . 32

8 Context-Free Grammars 34
8.1 Motivation . . . 34
8.2 Definitions . . . 34
8.3 Derivations and parse trees . . . 34
8.4 Ambiguity . . . 35
8.5 Precedence and associativity . . . 35
8.6 The WLP4 grammar (excerpt) . . . 36
8.7 Parsing as a problem . . . 36
8.8 Summary . . . 37

9 Top-Down Parsing 38
9.1 Idea . . . 38
9.2 Stack-based top-down parsing . . . 38
9.3 Nullable, FIRST and FOLLOW . . . 38
9.3.1 Computing them (fixed-point iteration) . . . 39
9.4 A worked example . . . 39
9.5 When a grammar is not LL(1) . . . 40
9.5.1 Left recursion . . . 40
9.5.2 Common prefixes: left factoring . . . 41
9.5.3 Limits . . . 41
9.6 Recursive descent . . . 41
9.7 Summary . . . 42

10 Bottom-Up Parsing 43
10.1 Idea . . . 43
10.2 Viable prefixes and items . . . 43
10.3 Building the LR(0) automaton . . . 44
10.4 The LR parsing algorithm . . . 44
10.5 Conflicts . . . 45
10.6 SLR(1) . . . 45
10.7 LR(1) and LALR(1) . . . 45
10.8 Parsing WLP4 . . . 46
10.9 Summary . . . 46

11 Context-Sensitive Analysis 47
11.1 Beyond context-free . . . 47
11.2 The WLP4 language . . . 47
11.3 Symbol tables . . . 48
11.4 Type checking . . . 49
11.4.1 Expressions . . . 49
11.4.2 Statements and tests . . . 49
11.4.3 Declarations and procedures . . . 49
11.5 Attribute grammars (brief) . . . 50
11.6 Errors and pitfalls . . . 50
11.7 Summary . . . 50

12 Code Generation **51**
12.1 Overview . . . 51
12.2 Conventions . . . 51
12.3 Variables and the stack frame . . . 52
12.4 Expressions . . . 52
12.4.1 Constants and variables . . . 52
12.4.2 Binary operators . . . 52
12.4.3 Pointer arithmetic . . . 53
12.4.4 Pointers: `*`, `&`, and assignment through pointers . . . 53
12.5 Tests . . . 53
12.6 Control flow . . . 54
12.6.1 Printing . . . 54
12.7 Procedures . . . 54
12.8 Dynamic memory: `new` and `delete` . . . 55
12.9 Putting it together . . . 56
12.10Summary . . . 57

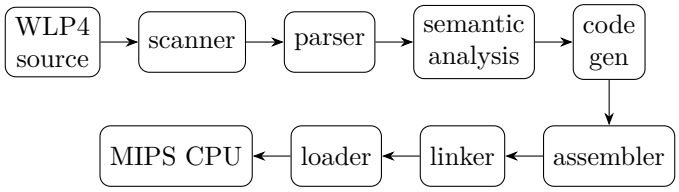
13 Optimization and Memory Management **58**
13.1 Optimization . . . 58
13.1.1 Constant folding and propagation . . . 58
13.1.2 Common subexpression elimination . . . 58
13.1.3 Dead code elimination . . . 58
13.1.4 Strength reduction and peephole optimization . . . 59
13.1.5 Register allocation . . . 59
13.1.6 Procedure-level optimizations . . . 59
13.2 Memory management . . . 60
13.2.1 Explicit allocation with a free list . . . 60
13.2.2 The buddy system . . . 60
13.3 Garbage collection . . . 61
13.3.1 Reference counting . . . 61
13.3.2 Mark and sweep . . . 61
13.3.3 Copying collection . . . 61
13.4 Summary . . . 62

Chapter 1

Introduction and Data Representation

1.1 What this course is about

CS 241 answers the question: *what actually happens between typing a program and the machine running it?* We build, piece by piece, the tool chain that turns source code into something a CPU executes:



The course is "bottom-up" in the first half (bits → machine code → assembly → assembler → linker/loader) and "top-down" in the second half (formal languages → scanning → parsing → semantic analysis → code generation). The word *sequential* in the title means we study programs that execute one instruction after another (no concurrency).

1.2 Bits, bytes and words

Definition 1.1. A **bit** is a 0 or 1. A **byte** is 8 bits; a **nibble** is 4 bits. A **word** is the machine's natural unit of data; on our MIPS machine a word is 32 bits (4 bytes).

A byte can hold $2^8 = 256$ distinct values; a word holds $2^{32} \approx 4.29 \times 10^9$. An n-bit sequence has 2^n possible values.

Key idea: bits have no inherent meaning

A sequence of bits is just a sequence of bits. Whether `11111111` means 255, -1, the character ÿ in Latin-1, or part of an instruction depends entirely on how it is *interpreted*. The same word in memory can be read as an unsigned integer, a signed integer, an ASCII string, or a MIPS instruction. The CPU only "knows" a word is an instruction because the program counter happens to point at it.

1.3 Number bases

A number written $d_{n-1} \dots d_1 d_0$ in base b represents

$$\sum_{i=0}^{n-1} d_i \, b^i, \qquad 0 \le d_i < b.$$

1.3.1 Binary and hexadecimal

Binary is base 2. Hexadecimal (hex) is base 16 using digits `0`–`9` and `a`–`f`. Each hex digit corresponds to exactly one nibble, so converting between binary and hex is just grouping bits in fours from the right:

$$\underbrace{1010}_{\texttt{a}}\ \underbrace{0011}_{\texttt{3}}\ \underbrace{1111}_{\texttt{f}}\ \underbrace{0000}_{\texttt{0}} = \texttt{0xa3f0}.$$

We write hex with a `0x` prefix. A 32-bit word is exactly 8 hex digits.

bin	hex	dec	bin	hex	dec
0000	0	0	1000	8	8
0001	1	1	1001	9	9
0010	2	2	1010	a	10
0011	3	3	1011	b	11
0100	4	4	1100	c	12
0101	5	5	1101	d	13
0110	6	6	1110	e	14
0111	7	7	1111	f	15

1.3.2 Decimal to binary

Two standard methods:

1. **Repeated division**: divide by 2, record remainders; read them bottom-to-top.
2. **Subtract powers**: find the largest power of 2 that fits, subtract, repeat.

Example 1.2. 38: $38/2 = 19\ r0$, $19/2 = 9\ r1$, $9/2 = 4\ r1$, $4/2 = 2\ r0$, $2/2 = 1\ r0$, $1/2 = 0\ r1$. Reading remainders upward: $100110_2 = \texttt{0x26}$. Check: $32 + 4 + 2 = 38$.

1.4 Unsigned integers

An n-bit unsigned integer uses the plain binary interpretation, covering $[0, 2^n - 1]$. Arithmetic is performed modulo 2^n: carries out of the top bit are lost (*overflow* wraps around). For $n = 32$, $\texttt{0xffffffff} + 1 = 0$.

1.5 Signed integers

1.5.1 Sign-magnitude (and why we do not use it)

Use the top bit as a sign and the rest as magnitude. Problems: two zeros ($+0$, -0), and addition needs case analysis on signs.

1.5.2 One's complement

Negate by flipping every bit. Still two zeros, and addition needs an "end-around carry".

1.5.3 Two's complement

Definition 1.3. In n-bit **two's complement**, bit pattern $b_{n-1} \dots b_0$ represents

$$-b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i.$$

Equivalently, interpret as unsigned u; if $u \geq 2^{n-1}$ the value is $u - 2^n$. The range is $[-2^{n-1}, 2^{n-1}-1]$.

So the top bit has *negative* weight. For 8 bits the range is $[-128, 127]$; for 32 bits it is $[-2^{31}, 2^{31}-1] = [-2147483648, 2147483647]$.

Negation in two's complement

To compute $-x$: flip all bits and add 1. This works because $x + \bar{x} = 2^n - 1$, so $\bar{x} + 1 = 2^n - x \equiv -x \pmod{2^n}$.

Example 1.4. 8-bit -38: $38 = 00100110$; flip $\to$ 11011001; add 1 $\to$ 11011010 = `0xda`. Check: $-128 + 64 + 16 + 8 + 2 = -38$.

Why two's complement wins. Addition, subtraction and (low-order) multiplication are *the same bit operations* for signed and unsigned numbers: everything is just arithmetic mod 2^n. Only comparisons, division, widening and overflow detection differ. That is why MIPS has `slt` vs. `sltu`, `div` vs. `divu`, `mult` vs. `multu`, but only one `add`.

Pitfalls.

- -2^{n-1} has no positive counterpart: negating `0x80000000` gives itself.
- Signed overflow: adding two positives can produce a negative (e.g. $127 + 1 = -128$ in 8 bits). Overflow happened iff both operands have the same sign and the result has the other sign.
- Comparing: `0xffffffff` is the largest unsigned word but signed it is -1.

1.5.4 Sign extension

To widen an n-bit two's complement number to $m > n$ bits, copy the sign bit into all new positions. E.g. 16-bit `0xfffe` (-2) becomes 32-bit `0xfffffffe`. Unsigned values are *zero-extended* instead. This matters for MIPS immediates: the 16-bit offsets in `lw`, `sw`, `beq`, `bne` are sign-extended.

1.6 Characters: ASCII

ASCII is a 7-bit code (stored in a byte) mapping integers to characters. Useful values:

character	decimal	hex
newline `\n`	10	`0x0a`
space	32	`0x20`
`'0'`	48	`0x30`
`'A'`	65	`0x41`
`'a'`	97	`0x61`

Digit d is character $d + 48$; lowercase is uppercase $+32$. To print an integer in MIPS we must convert each decimal digit to its ASCII code before writing it out.

1.7 Bit operations and shifts

Logical operations act bitwise: AND masks bits off, OR sets bits, XOR toggles them. A left shift by k multiplies by $2^k \pmod{2^n}$. A logical right shift fills with 0 (unsigned division by 2^k); an arithmetic right shift fills with the sign bit (signed division rounding toward $-\infty$). Our CS 241 MIPS subset has no shift instructions, so we emulate them with `mult`/`div` by powers of two.

1.8 Endianness and byte addressing

Memory is an array of bytes, each with an address. A word occupies four consecutive bytes, and on MIPS words must be *aligned*: their address is a multiple of 4. *Big-endian* stores the most significant byte at the lowest address; *little-endian* stores the least significant byte first. The CS 241 MIPS emulator is big-endian, so `0x12345678` at address `0x100` has `0x12` at `0x100`.

1.9 The stored-program idea

The machine has a CPU containing registers, an ALU, a program counter (PC) and an instruction register (IR), connected to RAM. Programs live in RAM alongside data (von Neumann architecture). The CPU repeatedly runs the *fetch–execute cycle*:

```
PC = 0;
while (true) {
    IR = MEM[PC];   // fetch
    PC = PC + 4;    // advance BEFORE executing
    execute(IR);    // decode and execute
}
```

Note that PC is incremented *before* the instruction executes; this is why branch offsets are relative to the address of the *next* instruction.

1.10 Summary

- Bits only acquire meaning from interpretation; the same word can be unsigned, signed, text or code.
- Hex is a compact notation for binary: one hex digit per four bits, eight per word.
- Two's complement: top bit has weight -2^{n-1}; negate by flip-and-add-one; add/sub are identical for signed and unsigned, comparisons and division are not.

- Sign-extend signed values, zero-extend unsigned ones.
- MIPS words are 32 bits, word-aligned and big-endian; the CPU runs fetch / increment PC / execute.

Chapter 2

MIPS Machine Language

2.1 The machine model

CS 241 uses a simplified 32-bit MIPS machine:

- 32 general-purpose registers `$0`–`$31`, each one word.
- Two special registers `hi` and `lo` that receive the results of multiplication and division.
- A program counter `PC` and an instruction register `IR` (not directly accessible).
- Byte-addressed RAM; all loads and stores are whole, word-aligned words.

2.1.1 Register conventions

Most registers are general purpose, but a few have fixed roles in CS 241:

register	role
`$0`	always 0; writes to it are discarded
`$1`, `$2`	inputs from the loader (`mips.twoints`, `mips.array`)
`$3`	by convention the return value of the program / of procedures
`$29`	frame pointer (used later by our WLP4 compiler)
`$30`	stack pointer; initialised by the loader to the end of usable memory
`$31`	return address; on start it holds the address that returns control to the loader

Ending a program

A program is started like a procedure call: the loader sets `$31` to a special return address and jumps to address 0. The program terminates by executing `jr $31`. If you overwrite `$31` (e.g. with `jalr`) you must save and restore it first.

2.2 The instruction set

Every instruction is exactly one 32-bit word. The CS 241 subset contains 18 instructions plus the `.word` directive. Notation: $\$d, \$s, \$t$ are registers, i is a 16-bit immediate.

instruction	assembly	meaning
add	`add $d, $s, $t`	$\$d \leftarrow \$s + \$t$
sub	`sub $d, $s, $t`	$\$d \leftarrow \$s - \$t$
mult	`mult $s, $t`	$hi{:}lo \leftarrow \$s \times \$t$ (signed)
multu	`multu $s, $t`	$hi{:}lo \leftarrow \$s \times \$t$ (unsigned)
div	`div $s, $t`	$lo \leftarrow \$s/\$t,\ hi \leftarrow \$s \bmod \$t$ (signed)
divu	`divu $s, $t`	same, unsigned
mfhi	`mfhi $d`	$\$d \leftarrow hi$
mflo	`mflo $d`	$\$d \leftarrow lo$
lis	`lis $d`	$\$d \leftarrow \mathrm{MEM}[PC];\ PC \leftarrow PC + 4$
lw	`lw $t, i($s)`	$\$t \leftarrow \mathrm{MEM}[\$s + i]$
sw	`sw $t, i($s)`	$\mathrm{MEM}[\$s + i] \leftarrow \$t$
slt	`slt $d, $s, $t`	$\$d \leftarrow (\$s < \$t)\ ?\ 1 : 0$ (signed)
sltu	`sltu $d, $s, $t`	same, unsigned
beq	`beq $s, $t, i`	if $\$s = \$t$ then $PC \leftarrow PC + 4i$
bne	`bne $s, $t, i`	if $\$s \neq \$t$ then $PC \leftarrow PC + 4i$
jr	`jr $s`	$PC \leftarrow \$s$
jalr	`jalr $s`	$tmp \leftarrow \$s;\ \$31 \leftarrow PC;\ PC \leftarrow tmp$
.word	`.word i`	a literal 32-bit word i (not an instruction)

Remarks:

- In `beq`/`bne` the PC has *already* been incremented, so the offset i counts words relative to the instruction after the branch. $i = 0$ is a no-op; $i = -1$ is an infinite loop.
- `lis` ("load immediate and skip") is a CS 241 invention: it loads the *next word* of the program into $\$d$ and skips over it. It is always followed by a `.word`. This is how we get arbitrary 32-bit constants into registers.
- `mult` produces a 64-bit product; the high word goes to `hi`, low to `lo`. For small numbers you only need `mflo`.
- `div` rounds toward zero; the remainder has the sign of the dividend.
- `jalr` reads $\$s$ before writing `$31`, so `jalr $31` is well defined.

2.3 Instruction encodings

There are two formats used by our subset.

Register format (R-type). Opcode 0, operation selected by the low 6 bits (the *function* field):

000000	sssss	ttttt	ddddd	00000	ffffff
6 bits	5	5	5	5	6

Immediate format (I-type). A 6-bit opcode, two registers, a 16-bit two's complement immediate:

oooooo	sssss	ttttt	iiii iiii iiii iiii
6 bits	5	5	16

instruction	fields used	opcode / funct	32-bit pattern
add	s,t,d	funct `0x20`	`0000 00ss ssst tttt dddd d000 0010 0000`
sub	s,t,d	funct `0x22`	`0000 00ss ssst tttt dddd d000 0010 0010`
mult	s,t	funct `0x18`	`0000 00ss ssst tttt 0000 0000 0001 1000`
multu	s,t	funct `0x19`	`0000 00ss ssst tttt 0000 0000 0001 1001`
div	s,t	funct `0x1a`	`0000 00ss ssst tttt 0000 0000 0001 1010`
divu	s,t	funct `0x1b`	`0000 00ss ssst tttt 0000 0000 0001 1011`
mfhi	d	funct `0x10`	`0000 0000 0000 0000 dddd d000 0001 0000`
mflo	d	funct `0x12`	`0000 0000 0000 0000 dddd d000 0001 0010`
lis	d	funct `0x14`	`0000 0000 0000 0000 dddd d000 0001 0100`
slt	s,t,d	funct `0x2a`	`0000 00ss ssst tttt dddd d000 0010 1010`
sltu	s,t,d	funct `0x2b`	`0000 00ss ssst tttt dddd d000 0010 1011`
jr	s	funct `0x08`	`0000 00ss sss0 0000 0000 0000 0000 1000`
jalr	s	funct `0x09`	`0000 00ss sss0 0000 0000 0000 0000 1001`
lw	s,t,i	op `0x23`	`1000 11ss ssst tttt iiii iiii iiii iiii`
sw	s,t,i	op `0x2b`	`1010 11ss ssst tttt iiii iiii iiii iiii`
beq	s,t,i	op `0x04`	`0001 00ss ssst tttt iiii iiii iiii iiii`
bne	s,t,i	op `0x05`	`0001 01ss ssst tttt iiii iiii iiii iiii`

Field order trap

In assembly, `add $d, $s, $t` lists the destination first, but in the encoding d comes *after* s and t. Similarly `lw $t, i($s)`: the base register s is encoded first.

Example 2.1 (Encoding `add $3, $1, $2`). $s = 1 = 00001$, $t = 2 = 00010$, $d = 3 = 00011$, funct $= 100000$:

$$\texttt{000000 00001 00010 00011 00000 100000}$$
$$= \texttt{0000 0000 0010 0010 0001 1000 0010 0000} = \texttt{0x00221820}.$$

Example 2.2 (Encoding `beq $1, $0, -3`). op $= 000100$, $s = 00001$, $t = 00000$, $i = -3 =$ `0xfffd` (16-bit two's complement):

$$\texttt{0001 0000 0010 0000 1111 1111 1111 1101} = \texttt{0x1020fffd}.$$

Example 2.3 (Decoding `0x8fc3fffc`). Binary: `100011 11110 00011 1111111111111100`. Opcode `0x23` is `lw`; $s = 30$, $t = 3$, $i =$ `0xfffc` $= -4$. So the instruction is `lw $3, -4($30)`.

In code, encoding is done with shifts and masks; masking the immediate with `0xffff` is essential so a negative immediate does not smear sign bits over the other fields:

```
uint32_t encodeI(int op, int s, int t, int i) {
    return (op << 26) | (s << 21) | (t << 16) | (i & 0xffff);
}
uint32_t encodeR(int s, int t, int d, int funct) {
    return (s << 21) | (t << 16) | (d << 11) | funct;
}
```

2.4 Loader conventions: `mips.twoints` and `mips.array`

The course provides two front-ends that load a machine-code program at address 0 and run it.

`mips.twoints` prompts for two integers and places them in `$1` and `$2`.

`mips.array` prompts for an array length and elements, stores the array somewhere in memory, and puts its *address* in `$1` and its *length* in `$2`. Element k is at address $\$1 + 4k$.

In both, `$30` points just past the end of usable memory (the initial top of the stack), `$31` holds the return address to the loader, and when the program executes `jr $31` the front-end prints the registers. The tools `cs241.wordasm` / `cs241.binasm` convert human-readable words to binary files so you can write machine code by hand before you have an assembler.

2.5 Memory-mapped I/O

There are no I/O instructions; two special addresses are wired to the terminal.

address	access	effect
`0xffff000c`	`sw`	write the low byte of the stored word to standard output
`0xffff0004`	`lw`	read one byte from standard input (-1 at end of file)

Example 2.4 (Print "Hi" and a newline)**.** Each `sw` to the output address prints one character (ASCII code in the low byte).

```
lis $1
.word 0xffff000c   ; output address
lis $2
.word 72           ; 'H'
sw $2, 0($1)
lis $2
.word 105          ; 'i'
sw $2, 0($1)
lis $2
.word 10           ; '\n'
sw $2, 0($1)
jr $31
```

2.6 First machine-language programs

Example 2.5 (Absolute value of `$1` into `$3`)**.** Run with `mips.twoints`; the result is left in `$3`.

```
add $3, $1, $0     ; $3 = $1
slt $2, $1, $0     ; $2 = ($1 < 0)
beq $2, $0, 1      ; if not negative, skip next instruction
sub $3, $0, $1     ; $3 = -$1
jr $31
```

Hand-assembled: `0x00201820`, `0x0020102a`, `0x10400001`, `0x00011822`, `0x03e00008`.

Example 2.6 (Sum $1 + 2 + \cdots + 13$)**.** A countdown loop; branch offsets are computed by hand.

```
lis $2
.word 13            ; counter
lis $1
.word 1             ; constant 1
add $3, $0, $0      ; sum = 0
add $3, $3, $2      ; sum += counter   <- loop top (address 0x14)
sub $2, $2, $1      ; counter--
bne $2, $0, -3      ; back to loop top
jr $31
```

The `bne` is at address `0x1c`; when it executes PC = `0x20`, and `0x20` + 4(−3) = `0x14`, the top of the loop.

2.7 Summary

- 32 registers; `$0`= 0, `$30` stack pointer, `$31` return address; `hi`/`lo` for mult/div.
- Two encoding formats: R-type (opcode 0 + funct) and I-type (opcode + 16-bit immediate). Destination comes last in R-type encodings.
- Branch offsets are in words, relative to PC+4; immediates are sign-extended.
- `lis` + `.word` loads 32-bit constants. Programs end with `jr $31`.
- I/O: `sw` to `0xffff000c` prints a byte; `lw` from `0xffff0004` reads one.

Chapter 3

Assembly Language Programming

3.1 From machine code to assembly

Writing hex by hand is error-prone, and branch offsets must be recomputed every time code is inserted. **Assembly language** is a textual, one-to-one representation of machine code with two conveniences:

- mnemonics and register names (`add $3, $1, $2` instead of `0x00221820`);
- **labels**: symbolic names for addresses, computed by the assembler.

Comments start with `;`. Each line has the form `[labels:]* [instruction or .word]? [; comment]?`.

3.2 Labels

A label definition `name:` binds `name` to the address of the next instruction (or `.word`) in the file. Several labels may precede the same instruction. A label may be used:

- in `beq`/`bne`, where the assembler computes the offset $(label - (PC_{\text{branch}} + 4))/4$;
- in `.word label`, which stores the label's *absolute* address (used with `lis` + `jalr` to call procedures).

Example 3.1 (Sum loop with labels)**.** The program from Chapter 2, now without hand-computed offsets:

```
    lis $2
    .word 13
    lis $1
    .word 1
    add $3, $0, $0
top:
    add $3, $3, $2
    sub $2, $2, $1
    bne $2, $0, top
    jr $31
```

3.3 Translating control structures

3.3.1 If-then-else

For `if (a < b) S1 else S2` with a in `$1`, b in `$2`:

```
    slt $5, $1, $2       ; $5 = a < b
    beq $5, $0, else     ; false -> else
    ; ... S1 ...
    beq $0, $0, endif    ; unconditional jump
else:
    ; ... S2 ...
endif:
```

`beq $0, $0, label` is the standard unconditional branch.

3.3.2 Comparisons

Only `slt`/`sltu` exist; derive the rest:

- $a > b$: `slt $5, $2, $1` (swap operands).
- $a \geq b$: compute $a < b$ and invert. With `$11` holding 1: `slt $5,$1,$2; sub $5,$11,$5`.
- $a \leq b$: invert $b < a$.
- $a = b$, $a \neq b$: use `beq`/`bne` directly, or `slts`: $(a < b) + (b < a)$ is 1 iff $a \neq b$.

3.3.3 Loops

A `while (cond) body` becomes a test at the top and a jump back at the bottom:

```
loop:
    ; compute cond into $5
    beq $5, $0, done
    ; ... body ...
    beq $0, $0, loop
done:
```

3.4 Arrays

With `mips.array`, `$1` is the array address and `$2` its length. Element i is at $\$1 + 4i$. Since `lw` takes only a constant offset, compute the address explicitly.

Example 3.2 (Maximum element of an array, result in `$3`)**.** Run with `mips.array` (assumes length ≥ 1):

```
    lis $4
    .word 4            ; word size
    lis $11
    .word 1
    lw  $3, 0($1)      ; max = a[0]
    add $5, $1, $0     ; p = &a[0]
    add $6, $2, $0     ; n remaining
```

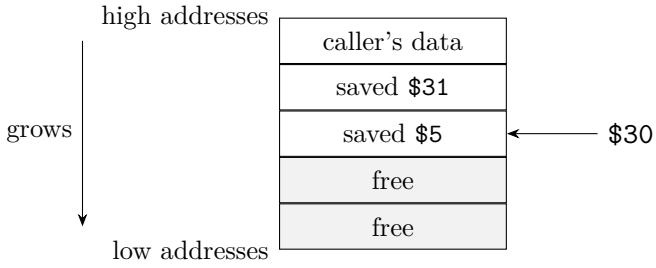
```
loop:
    lw  $7, 0($5)       ; x = *p
    slt $8, $3, $7      ; max < x ?
    beq $8, $0, skip
    add $3, $7, $0      ; max = x
skip:
    add $5, $5, $4      ; p++
    sub $6, $6, $11     ; n--
    bne $6, $0, loop
    jr $31
```

Pointer increment ("walk a pointer") is cheaper than recomputing $\$1 + 4i$ each time. To index directly: `mult $i, $4; mflo $t; add $t, $t, $1; lw $x, 0($t)`.

3.5 The stack

Registers are a shared global resource; procedures need private storage. The loader initialises `$30` to the end of memory and we use it as a **stack pointer**. The stack grows *downward*, and `$30` points to the last-pushed word (the top).

```
; push $5
sw  $5, -4($30)
sub $30, $30, $4      ; $4 = 4
; pop into $5
add $30, $30, $4
lw  $5, -4($30)
```



3.6 Procedures

A procedure is a labelled block of code that ends in `jr $31`. To call it:

```
    lis $5
    .word proc        ; absolute address of proc
    jalr $5           ; $31 = return address, jump to proc
```

The `$31` problem. `jalr` overwrites `$31`, which our own program needs to return to the loader. So every caller (including `main`) must save `$31` on the stack before `jalr` and restore it after:

```
    sw   $31, -4($30)     ; push $31
    lis  $31
    .word 4
```

```
    sub  $30, $30, $31
    lis  $5
    .word proc
    jalr $5
    lis  $31
    .word 4
    add  $30, $30, $31   ; pop $31
    lw   $31, -4($30)
```

(Here `$31` itself is reused as a temporary holding 4, since its real value is safely on the stack.)

3.6.1 Calling conventions

A **calling convention** is the contract between caller and callee. The convention we use in CS 241 assignments:

- Arguments are passed in registers (documented per procedure) or on the stack.
- The return value is placed in `$3`.
- **Callee-save**: the procedure must restore every register it modifies, except its documented output(s). It pushes them at entry and pops them before `jr $31`.
- The caller saves `$31` around `jalr`.

The alternative, *caller-save*, makes the caller save any register it cares about before calling. Real ABIs mix the two (some registers callee-saved, some caller-saved).

Document your procedures

For each procedure write a header comment: what it does, which registers hold inputs, which hold outputs, and which (if any) are clobbered. Bugs in assembly are almost always convention violations: a forgotten restore, a stack pointer that is off by 4, or `$31` not saved.

Example 3.3 (Procedure: sum of 1..`$1` in `$3`)**.** A callee-save procedure: it restores every register it uses except its output.

```
; sum1toN: $3 = 1 + 2 + ... + $1 ($1 >= 1)
; registers modified: $3 (output) -- $1, $2 preserved
sum1toN:
    sw  $1, -4($30)      ; save $1
    sw  $2, -8($30)      ; save $2
    lis $2
    .word 8
    sub $30, $30, $2     ; reserve 2 words
    lis $2
    .word 1
    add $3, $0, $0
sloop:
    add $3, $3, $1
    sub $1, $1, $2
    bne $1, $0, sloop
    lis $2
    .word 8
```

```
    add $30, $30, $2     ; release 2 words
    lw  $1, -4($30)
    lw  $2, -8($30)
    jr  $31
```

3.7 Stack frames and recursion

Recursion works automatically once every procedure saves `$31` and its registers on the stack: each activation gets its own region, its **stack frame** (activation record). A frame typically contains saved registers, the saved return address, and local variables.

Example 3.4 (Recursive factorial)**.** Each activation pushes a three-word frame holding `$1`, `$2` and `$31`.

```
; fact: $3 = $1!   (uses $1 as input; preserves all but $3)
fact:
    sw  $1,  -4($30)
    sw  $2,  -8($30)
    sw  $31, -12($30)
    lis $2
    .word 12
    sub $30, $30, $2     ; push frame of 3 words
    lis $3
    .word 1              ; base result
    lis $2
    .word 1
    slt $2, $2, $1       ; $2 = (1 < n)
    beq $2, $0, fdone    ; n <= 1 -> return 1
    lis $2
    .word 1
    sub $1, $1, $2       ; n - 1
    lis $2
    .word fact
    jalr $2              ; $3 = (n-1)!
    lw  $1, 8($30)       ; reload original n from frame
    mult $3, $1
    mflo $3              ; $3 = n * (n-1)!
fdone:
    lis $2
    .word 12
    add $30, $30, $2     ; pop frame
    lw  $1,  -4($30)
    lw  $2,  -8($30)
    lw  $31, -12($30)
    jr  $31
```

After pushing, the saved $1 is at offset 8 from the new `$30` (frame is $30+0: `$31`, $30+4: `$2`, $30+8: `$1`).

Recursion depth. Each call uses a fixed amount of stack, so depth d uses $\Theta(d)$ stack words. Infinite recursion eventually runs `$30` into the program or heap and crashes.

3.8 Input and output

Example 3.5 (Echo standard input to standard output)**.** Read bytes until EOF (-1), writing each one back out.

```
    lis $1
    .word 0xffff0004     ; input
    lis $2
    .word 0xffff000c     ; output
    lis $4
    .word -1             ; EOF marker
loop:
    lw  $3, 0($1)        ; read a byte
    beq $3, $4, end
    sw  $3, 0($2)        ; write it
    beq $0, $0, loop
end:
    jr $31
```

Printing a number. To print a signed integer: print `-` and negate if negative; then repeatedly `div` by 10, pushing remainders (`mfhi`) on the stack until the quotient is 0; finally pop each digit, add 48, and store it to `0xffff000c`. Pitfalls: the number 0 must print one digit, and -2^{31} cannot be negated (handle it via unsigned division or by working with negative remainders).

3.9 Summary

- Assembly = machine code with mnemonics, comments and labels; the assembler computes branch offsets and label addresses.
- `if`/`while` translate to `slt` + `beq`/`bne`; `beq $0,$0,L` is an unconditional jump.
- The stack grows down from `$30`; push = store then decrement, pop = increment then load.
- Call with `lis`/`.word`/`jalr`; save `$31` first. Return with `jr $31`.
- Calling convention: callee restores everything except outputs; results in `$3`.
- Recursion needs nothing extra: each activation's state lives in its own stack frame.

Chapter 4

Assemblers

4.1 The job of an assembler

An **assembler** translates assembly language to machine code. Since assembly is (almost) one-to-one with machine code, the interesting parts are:

1. **Analysis**: break the text into tokens, check each line is well formed, and collect label definitions.
2. **Synthesis**: produce the 32-bit word for each instruction, resolving label references.

Output is binary written to standard output (four bytes per word, big-endian); errors are reported on standard error, and on any error no (partial) output should be considered valid.

4.2 Scanning the input

The course supplied a scanner (in C++ and Racket) that splits each line into tokens with kinds such as `ID`, `LABEL` (`foo:`), `REG` (`$12`), `INT` (`-17`), `HEXINT` (`0x1f`), `COMMA`, `LPAREN`, `RPAREN`, `DOTID` (`.word`) and `NEWLINE`. How scanners work in general is the subject of Chapter 7; here we simply consume the token stream.

4.3 The forward-reference problem

A label may be used before it is defined:

```
    beq $1, $0, done     ; 'done' not seen yet!
    add $3, $3, $1
done:
    jr $31
```

When we reach the `beq` we do not know `done`'s address. Two solutions:

- **Two passes**: first pass computes all label addresses; second pass encodes.
- **One pass with backpatching**: emit a placeholder and record the location; patch it when the label is defined (requires buffering output).

CS 241 uses the two-pass approach.

4.4 Two-pass assembly

4.4.1 Pass 1: analysis and the symbol table

Maintain a *location counter* `PC`, starting at 0.

```
std::map<std::string, uint32_t> symtab;
uint32_t pc = 0;
for (auto &line : lines) {
    size_t k = 0;
    while (k < line.size() && line[k].kind == LABEL) {
        std::string name = labelName(line[k]);      // strip ':'
        if (symtab.count(name)) error("duplicate label " + name);
        symtab[name] = pc;
        ++k;
    }
    if (k < line.size()) {               // an instruction or .word
        checkSyntax(line, k);            // operand kinds, counts, commas
        pc += 4;
    }
}
```

Key points: a label alone on a line (or followed only by a comment) does not advance `PC`; labels mark the *next* instruction even if that is several lines later.

4.4.2 Pass 2: synthesis

Walk the lines again with a fresh `PC`. For each instruction, look up the opcode/funct, check operand ranges, substitute labels, encode with shifts and masks, and write four bytes.

```
void outWord(uint32_t w) {
    putchar((w >> 24) & 0xff);
    putchar((w >> 16) & 0xff);
    putchar((w >> 8) & 0xff);
    putchar(w & 0xff);
}
// branch: label resolved relative to PC + 4
int32_t off = ((int32_t)symtab.at(label) - (int32_t)(pc + 4)) / 4;
if (off < -32768 || off > 32767) error("branch out of range");
outWord((op << 26) | (s << 21) | (t << 16) | (off & 0xffff));
```

Example 4.1. Addresses are shown in the comments; the arrows give the encoded values.

```
main:  lis $2          ; 0x00
       .word end       ; 0x04  -> 0x00000014
loop:  beq $1, $0, end ; 0x08  -> offset (0x14 - 0x0c)/4 = 2
       sub $1, $1, $2  ; 0x0c
       beq $0, $0, loop; 0x10  -> offset (0x08 - 0x14)/4 = -3
end:   jr $31          ; 0x14
```

Symbol table: `main`↦0x00, `loop`↦0x08, `end`↦0x14.

4.5 Error checking

A good assembler rejects every malformed program. Typical checks:

category	examples
syntax	wrong number of operands; missing commas/parentheses; unknown mnemonic; a label after an instruction on the same line
registers	register number outside 0–31
immediates	`lw`/`sw`/`beq`/`bne` decimal immediate outside $[-32768, 32767]$; hex immediate above `0xffff`
`.word`	decimal outside $[-2^{31}, 2^{32}-1]$; hex above `0xffffffff`
labels	duplicate definition; use of an undefined label; computed branch offset out of 16-bit range

Ranges

A `.word` accepts any value that fits in 32 bits under *either* interpretation, so $[-2^{31}, 2^{32}-1]$. A 16-bit immediate in decimal must be in $[-2^{15}, 2^{15}-1]$, but a hex immediate may be up to `0xffff` (it is just a bit pattern). Be careful to parse large integers into a 64-bit type before range checking, or overflow will hide errors.

Error output convention in CS 241: print a message containing `ERROR` to standard error and stop. A useful structure is to throw an exception from deep inside parsing and catch it in `main`.

4.6 Assembler design notes

- Represent each instruction format generically: a table maps mnemonic to (format, opcode/-funct, operand pattern), e.g. `add`$\mapsto$(R, 0x20, `reg,reg,reg`). This avoids eighteen near-identical cases.
- Pass 1 can store a parsed intermediate representation, so pass 2 need not re-check syntax.
- Running time is $\Theta(n \log n)$ with a balanced-tree map, or expected $\Theta(n)$ with a hash table, for n lines.
- The symbol table is often printed to standard error for debugging (the assignment asked for this).

4.7 Beyond the CS 241 subset

Real assemblers also support *pseudo-instructions* that expand into several real instructions (e.g. `li`, `move`, `blt`), directives for data and sections (`.data`, `.text`, `.ascii`), and expressions such as `label+8`. Pseudo instructions complicate pass 1 because one source line may produce a variable number of words.

4.8 Summary

- Forward references force either two passes or backpatching.
- Pass 1: walk lines, record label $\mapsto$ address in the symbol table, check syntax, advance PC by 4 per instruction or `.word`.

- Pass 2: encode each instruction; branch offset = $(label - (PC + 4))/4$; `.word label` stores the absolute address.
- Check everything: syntax, register numbers, immediate ranges (decimal vs. hex), duplicate and undefined labels.

Chapter 5

Linking, Loading and Relocation

5.1 Loaders

So far every program was assumed to start at address 0. In reality the operating system runs many programs and chooses where each one goes. The **loader** is the part of the OS that:

1. finds free memory at some address α;
2. copies the program's words into $\text{MEM}[\alpha], \text{MEM}[\alpha+4], \ldots$;
3. sets up registers (`$30`, `$31`, any inputs) and jumps to α.

```
// naive loader
alpha = findFreeRAM(n);
for (i = 0; i < n; i++)
    MEM[alpha + 4*i] = file[i];
$30 = alpha + n*4 + stackSize;   // or end of memory
$31 = returnToOS;
PC = alpha;
```

The OS itself is a program that is already loaded; conceptually it runs in a loop: read a program name, load it, `jalr` to it, and continue when it returns.

5.2 Relocation

The problem. Machine code assembled for address 0 is wrong at address α wherever it contains an *absolute address*. In our subset that means exactly the `.word label` lines: `.word foo` contains the address of `foo` assuming the program starts at 0. Branch offsets are PC-relative, so they are fine.

Solution. The loader adds α to every word that holds an address. But a word in a binary file looks just like any other word, so the assembler must tell the loader *which* words to adjust. That information is stored in an **object file** format. In CS 241 we use **MERL** (MIPS Executable Relocatable Linkable).

5.3 The MERL format

A MERL file consists of a 3-word header, the code, and a table of entries:

offset	contents	meaning
`0x00`	`0x10000002`	cookie: the instruction `beq $0, $0, 2`
`0x04`	*endModule*	total length of the file in bytes
`0x08`	*endCode*	length of header + code in bytes
`0x0c`	code ...	the assembled program
endCode	table ...	relocation and external symbol entries

The cookie is clever: it is a valid instruction that skips the next two words, so a MERL file loaded at address 0 can be run directly, jumping over *endModule* and *endCode*.

Table entries:

type	format	meaning
REL	`0x01`, *loc*	word at *loc* holds a relocatable address
ESR	`0x11`, *loc*, *len*, $c_1, \ldots, c_{len}$	external symbol reference: word at *loc* should hold the address of the imported symbol named $c_1 \ldots c_{len}$ (one ASCII char per word)
ESD	`0x05`, *loc*, *len*, $c_1, \ldots, c_{len}$	external symbol definition: this module exports a symbol with that name at address *loc*

All locations are relative to the beginning of the MERL file *including* the 12-byte header, i.e. the code is assembled as though it starts at address `0x0c`.

Example 5.1. Source:

```
    lis $3
    .word ten
    jr $31
ten: .word 10
```

MERL output (each line one word):

```
beq $0, $0, 2       ; 0x00 cookie
.word 0x24          ; 0x04 endModule
.word 0x1c          ; 0x08 endCode
lis $3              ; 0x0c
.word 0x18          ; 0x10 address of ten (0x0c + 0x0c)
jr $31              ; 0x14
.word 10            ; 0x18 ten
.word 0x01          ; 0x1c REL entry
.word 0x10          ; 0x20 location to relocate
```

5.4 A relocating loader

```
read cookie, endMod, endCode;
codeLen = endCode - 12;
alpha = findFreeRAM(codeLen + stack);
for (i = 0; i < codeLen; i += 4)
    MEM[alpha + i] = readWord();
i = endCode;
while (i < endMod) {
    format = readWord();
    if (format == 1) {                // REL
        rel = readWord();
        MEM[alpha + rel - 12] += alpha - 12;
        i += 8;
    } else error("unknown format");
}
```

Why $\alpha - 12$? The stored address already includes the header offset of 12 (code was assembled starting at `0x0c`), but the loader places the first code word (not the header) at α. So the true address is $stored - 12 + \alpha$.

Position-independent code

Code containing no absolute addresses (only PC-relative branches) is **position-independent** and needs no relocation. Hidden absolute addresses that the assembler cannot see (e.g. computing an address by arithmetic on a constant) will break under relocation, so always produce addresses with `.word label`.

5.5 Linking

Large programs are split across files, and libraries (like the `print` routine or `alloc.merl` used by our compiler) are assembled separately. A **linker** combines object files into one, resolving references between them.

Directives.

- `.import name` declares that `name` is defined in another module. Using it in `.word name` generates an ESR entry (with value 0 in the code).
- `.export name` makes a label defined in this module visible to others (generates an ESD entry).

An assembler for MERL must check: an imported name must not also be a local label; an exported name must be defined; a label used must be either defined or imported.

5.5.1 Linking algorithm

To link m_1 and m_2 (producing a module in which m_2's code follows m_1's):

1. Check no symbol is exported by both (duplicate definition $\Rightarrow$ error).
2. Let $\alpha = endCode(m_1) - 12$. Relocate m_2: add α to every location in m_2's table, and add α to each word in m_2's code marked REL.

3. For each ESR in m_1 whose name is ESD-exported by m_2: write the ESD's address into the referenced word and change the ESR into a REL (it is now an internal address, still needing relocation at load time). Symmetrically for ESRs in m_2 resolved by m_1's ESDs.

4. Concatenate: header, code of m_1, code of m_2, combined tables. Recompute *endCode* and *endModule*.

ESRs that remain unresolved stay as ESRs, so linking can happen in stages; a final executable should have none left.

```
alpha = m1.endCode - 12;
for (e : m2.table) e.loc += alpha;
for (e : m2.table) if (e.type == REL) m2.code[e.loc] += alpha;
for (d : m2.ESDs) d.addr += alpha;   // (same as loc for ESD)
for (r : m1.ESRs) if (m2 exports r.name) {
    m1.code[r.loc] = addressOf(r.name); r.type = REL; }
for (r : m2.ESRs) if (m1 exports r.name) {
    m2.code[r.loc] = addressOf(r.name); r.type = REL; }
```

5.6 Static vs. dynamic linking

In the course we do *static* linking: all modules are merged into one file before running. Real systems also use *dynamic* linking (shared libraries, `.so`/`.dll`), where references are resolved at load time or on first call, so library code is shared among processes and can be updated without relinking. Real object formats (ELF, COFF, Mach-O) carry the same kinds of information as MERL: code, relocation records, and symbol tables.

5.7 Summary

- The loader picks an address α, copies code there, and must relocate absolute addresses.
- In our subset, only `.word label` produces absolute addresses; branches are relative.
- MERL = header (cookie `0x10000002`, endModule, endCode) + code + table of REL, ESR, ESD entries; locations include the 12-byte header.
- Relocation at load: MEM$[\alpha + loc - 12]$ += $\alpha - 12$.
- Linking: offset the second module, resolve ESRs against ESDs (turning them into RELs), concatenate, fix the header; duplicate exports are an error.

Chapter 6

Formal Languages and Regular Languages

6.1 Why formal languages?

The second half of the course builds a compiler. To recognise and analyse programs we need a precise notion of what a "valid program" is. Formal language theory gives us a hierarchy of language classes, each with a matching recogniser:

class	described by	recognised by
finite	a list	lookup
regular	regular expressions	DFAs / NFAs
context-free	context-free grammars	pushdown automata (parsers)
context-sensitive	context-sensitive grammars	linear-bounded automata
recursive / r.e.	unrestricted grammars	Turing machines

Compilers use regular languages for *scanning* (tokens), context-free languages for *parsing* (structure), and ad-hoc context-sensitive checks for *semantic analysis* (types, declarations).

6.2 Basic definitions

Definition 6.1. An **alphabet** Σ is a finite, non-empty set of symbols. A **word** (string) over Σ is a finite sequence of symbols from Σ; ε denotes the empty word, and $|w|$ the length of w. Σ^* is the set of all words over Σ. A **language** is any subset $L \subseteq \Sigma^*$.

Note the distinctions: $\emptyset$ is the empty language, $\{\varepsilon\}$ is the language containing only the empty word; these are different.

Operations on languages.

- Union $L_1 \cup L_2$.
- Concatenation $L_1 L_2 = \{xy : x \in L_1, y \in L_2\}$.
- Powers: $L^0 = \{\varepsilon\}$, $L^{n+1} = L^n L$.
- Kleene star $L^* = \bigcup_{n \geq 0} L^n$; $L^+ = LL^*$.

6.3 Regular languages

Definition 6.2. The **regular languages** over Σ are the smallest class such that $\emptyset$, $\{\varepsilon\}$ and $\{a\}$ (for each $a \in \Sigma$) are regular, and that is closed under union, concatenation and Kleene star.

All finite languages are regular. $\{a^n b^n : n \geq 0\}$ is *not* regular (a finite automaton cannot count unboundedly); this is the classic motivation for context-free grammars.

6.3.1 Regular expressions

A regular expression denotes a regular language:

regex	language
$\emptyset$	$\emptyset$
ε	$\{\varepsilon\}$
a	$\{a\}$
$E_1 \mid E_2$	$L(E_1) \cup L(E_2)$
$E_1 E_2$	$L(E_1)L(E_2)$
E^*	$L(E)^*$

Precedence: star binds tightest, then concatenation, then alternation.

Example 6.3.

- Binary strings with an even number of 1s: $(0^*10^*1)^*0^*$.
- C identifiers: $(\ell \mid _)(\ell \mid d \mid _)^*$ where ℓ = letters, d = digits.
- Decimal integers without leading zeros: $0 \mid (1|\ldots|9)(0|\ldots|9)^*$.

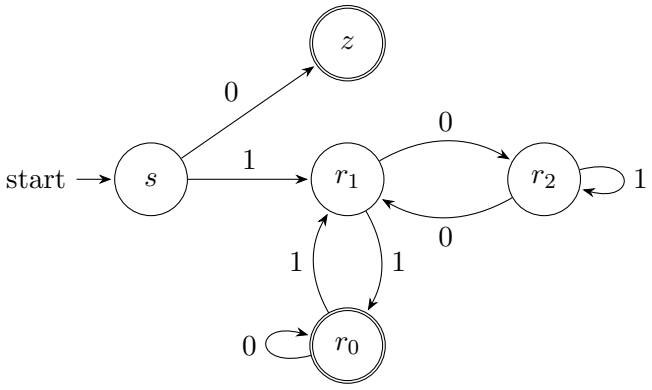
6.4 Deterministic finite automata

Definition 6.4. A **DFA** is a 5-tuple $(\Sigma, Q, q_0, A, \delta)$: alphabet Σ, finite set of states Q, start state $q_0 \in Q$, accepting states $A \subseteq Q$, and transition function $\delta : Q \times \Sigma \to Q$.

Extend δ to words: $\delta^*(q, \varepsilon) = q$ and $\delta^*(q, aw) = \delta^*(\delta(q, a), w)$. The DFA accepts w iff $\delta^*(q_0, w) \in A$, and $L(M) = \{w : \delta^*(q_0, w) \in A\}$.

Convention. We usually draw only useful transitions; any missing transition goes to an implicit non-accepting *error* (trap) state.

Example 6.5 (Binary numbers with no leading zeros that are divisible by 3)**.** Track the value mod 3: reading bit b maps remainder r to $(2r + b) \bmod 3$. State z means the input was exactly `0`; state r_i means a number with no leading zero and remainder i.



(Check: from r_2, bit 0 gives $4 \bmod 3 = 1$, bit 1 gives $5 \bmod 3 = 2$. From r_1: bit 0 gives 2, bit 1 gives 0. From r_0: bit 0 gives 0, bit 1 gives 1.)

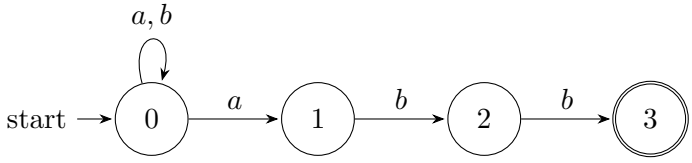
Implementation. A DFA is trivially simulated in $\Theta(|w|)$ time:

```
int state = start;
for (char c : w) {
    state = delta[state][c];
    if (state == ERROR) return false;
}
return accepting[state];
```

6.5 Nondeterministic finite automata

Definition 6.6. An **NFA** is like a DFA but $\delta : Q \times \Sigma \to 2^Q$: from a state, a symbol may lead to zero, one or several states. Define $\delta^*(S, \varepsilon) = S$ and $\delta^*(S, aw) = \delta^*(\bigcup_{q \in S} \delta(q, a), w)$. The NFA accepts w iff $\delta^*(\{q_0\}, w) \cap A \neq \emptyset$, i.e. *some* sequence of choices leads to acceptance.

NFAs are often much easier to design. Example: strings over $\{a, b\}$ ending in abb.



6.5.1 Simulating an NFA

Keep the set of current states:

```
std::set<int> S = {start};
for (char c : w) {
    std::set<int> T;
    for (int q : S) for (int r : delta[q][c]) T.insert(r);
    S = T;
}
return intersects(S, accepting);
```

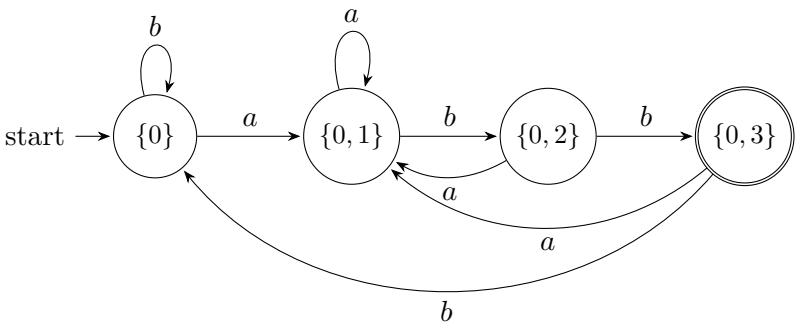
6.5.2 Subset construction (NFA → DFA)

Theorem 6.7. *For every NFA there is a DFA accepting the same language.*

Construction. DFA states are *sets* of NFA states. Start state $\{q_0\}$; from set S on a go to $\bigcup_{q\in S}\delta(q,a)$; a set is accepting iff it contains an accepting NFA state. Only reachable sets need be built. In the worst case the DFA has $2^{|Q|}$ states, but usually far fewer.

Example 6.8 (Subset construction for "ends in *abb*")**.** Starting from $\{0\}$ and building only the reachable sets:

DFA state	on a	on b
$\{0\}$	$\{0,1\}$	$\{0\}$
$\{0,1\}$	$\{0,1\}$	$\{0,2\}$
$\{0,2\}$	$\{0,1\}$	$\{0,3\}$
$\{0,3\}^*$	$\{0,1\}$	$\{0\}$



6.6 ε-NFAs

An **ε-NFA** additionally allows transitions labelled ε that may be taken without consuming input. The **ε-closure** $E(S)$ of a set S is all states reachable from S using only ε-moves (computed by DFS/BFS). Modify the definitions: start in $E(\{q_0\})$, and after each symbol take the ε-closure:

$$\delta^*(S,aw)=\delta^*\Big(E\big(\textstyle\bigcup_{q\in S}\delta(q,a)\big),w\Big).$$

Subset construction works the same way with closures, so ε-NFAs, NFAs and DFAs all recognise exactly the same languages.

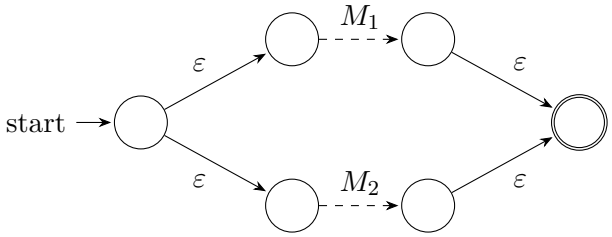
6.7 Kleene's theorem

Theorem 6.9 (Kleene)**.** *A language is regular if and only if it is accepted by some DFA.*

Regex ⇒ ε-NFA (Thompson's construction). By structural induction, build an ε-NFA with one start and one accepting state:

- $\emptyset$: start state, separate accepting state, no transitions.
- ε: a single state that is both start and accepting.

- a: start $\xrightarrow{a}$ accept.
- $E_1 \mid E_2$: new start with ε-edges to both machines' starts; both old accept states $\xrightarrow{\varepsilon}$ a new accept state.
- E_1E_2: ε-edge from M_1's accept to M_2's start.
- E^*: new start state that is also accepting, with ε to M's start; ε from M's accept back to the new start.



Thompson construction for $E_1 \mid E_2$.

DFA $\Rightarrow$ regex. By state elimination (or the dynamic-programming $R_{ij}^{(k)}$ construction): repeatedly remove a state, relabelling edges with regular expressions that account for paths through it, until only start and accept remain. CS 241 mentions this direction but does not drill it.

Closure properties. Regular languages are closed under union, concatenation, star (by definition), and also complement (swap accepting/non-accepting states of a *complete* DFA) and intersection (product construction, or De Morgan).

6.8 Summary

- Alphabet, word, language; $\emptyset \neq \{\varepsilon\}$.
- Regular languages: built from finite pieces with $\cup$, concatenation, *; written as regexes.
- DFA: one transition per state/symbol, linear-time recognition; missing edges go to an error state.
- NFA and ε-NFA: easier to build; convert with the subset construction (worst case 2^n states), using ε-closures.
- Kleene: regex $\Leftrightarrow$ ε-NFA $\Leftrightarrow$ NFA $\Leftrightarrow$ DFA.
- $\{a^n b^n\}$ is not regular, motivating context-free grammars.

Chapter 7

Scanning

7.1 The scanning problem

A **scanner** (lexer, tokenizer) is the first phase of a compiler. It reads a string of characters and splits it into **tokens**, each a pair (*kind*, *lexeme*):

$$\texttt{int x = 42;} \implies (\text{INT}, \texttt{int})\ (\text{ID}, \texttt{x})\ (\text{BECOMES}, =)\ (\text{NUM}, \texttt{42})\ (\text{SEMI}, ;)$$

Whitespace and comments are discarded. Each token kind is a regular language, so the language of all valid token sequences, $L = (T_1 \mid T_2 \mid \cdots \mid T_k)^*$ (with whitespace interleaved), is regular too.

Scanning is a decomposition problem

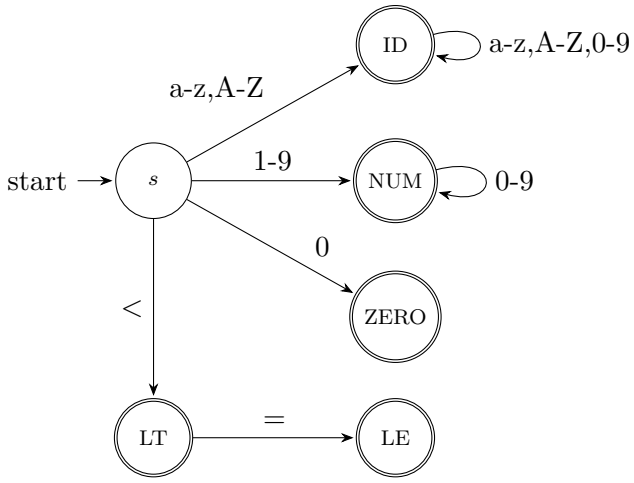
Recognising whether $w \in L^*$ is easy, but the scanner must also produce *a decomposition* of w into tokens. In general the decomposition is not unique: is `abc` one identifier or three? Is `>=` one token or `>` then `=`? We need a rule to choose.

7.2 Building a token-recognising DFA

1. Write a regular expression for each token kind.
2. Build an ε-NFA for their union, remembering for each accepting state which token kind it accepts.
3. Convert to a DFA by subset construction. An accepting DFA state may contain accepting NFA states for several kinds (e.g. keyword `int` vs. identifier); resolve by a priority order (keywords beat identifiers).

In practice (and in the CS 241 WLP4 scanner assignment) keywords are recognised as identifiers by the DFA, and afterwards a lookup checks whether the lexeme is a reserved word.

Example 7.1 (Partial WLP4 token DFA)**.** A fragment of the DFA (keywords are recognised as `ID` and looked up afterwards):



Note `ZERO` has no outgoing edges: in WLP4 `0` followed by a digit is an error, not a longer number.

7.3 Maximal munch

Definition 7.2. The **maximal munch** rule: at each step, choose the *longest* prefix of the remaining input that is a valid token.

General maximal munch algorithm. Run the DFA from the current position as far as possible, remembering the last position at which it was in an accepting state. When the DFA gets stuck (or the input ends), back up to that last accepting position, emit that token, and restart from there.

```
std::vector<Token> scan(const std::string &w) {
    std::vector<Token> out;
    size_t pos = 0;
    while (pos < w.size()) {
        int state = START;
        size_t i = pos, lastAcceptEnd = std::string::npos;
        int lastAcceptState = ERROR;
        while (i < w.size() && delta(state, w[i]) != ERROR) {
            state = delta(state, w[i++]);
            if (accepting(state)) { lastAcceptEnd = i; lastAcceptState = state; }
        }
        if (lastAcceptEnd == std::string::npos) throw ScanError(pos);
        out.push_back({kindOf(lastAcceptState), w.substr(pos, lastAcceptEnd - pos)});
        pos = lastAcceptEnd;          // back up to last accept
    }
    return out;
}
```

The backing up can cause $\Theta(n^2)$ worst-case time: e.g. if tokens are `a` and `a...ab` the scanner may read far ahead before failing and backing up, repeatedly. (A memoised version, due to Reps, achieves linear time.)

7.4 Simplified maximal munch

Definition 7.3. Simplified maximal munch: run the DFA until it gets stuck. If the current state is accepting, emit the token; otherwise report an error. Never back up.

```
while (pos < w.size()) {
    int state = START; size_t i = pos;
    while (i < w.size() && delta(state, w[i]) != ERROR)
        state = delta(state, w[i++]);
    if (!accepting(state)) throw ScanError(pos);
    emit(kindOf(state), w.substr(pos, i - pos));
    pos = i;
}
```

This runs in $\Theta(n)$ time and is what CS 241's WLP4 and MIPS-assembly scanners use. The price: some strings that *can* be decomposed into tokens are rejected.

Example 7.4 (Simplified maximal munch can fail)**.** Suppose tokens are $\{a,\ aab\}$ (alternatively: C++ `>>` vs. nested templates). On input aaa: the DFA reads a (accepting), then aa (not accepting, but still alive since it is a prefix of aab), then the third a kills it after aa. The state for aa is not accepting, so simplified munch reports an error. Yet $aaa = a \cdot a \cdot a$ is a valid decomposition, and full maximal munch would find it by backing up.

Example 7.5 (Maximal munch is not always what you want)**.** In C++03, `vector<vector<int>>` scanned `>>` as a single shift token, a famous annoyance (fixed in C++11 by a special parser rule). Similarly, `x---y` is scanned as `x -- - y`. Maximal munch is greedy, not a full search over decompositions.

7.5 Whitespace, comments, and token separation

In WLP4, whitespace and comments (`//` to end of line) are recognised as "tokens" by the DFA and then discarded. Some pairs of tokens must be separated by whitespace, because under maximal munch they would otherwise merge: `int x` vs. `intx`, or `42abc` (WLP4 treats a NUM immediately followed by a letter as an error). Numbers out of range (above $2^{31} - 1$) are rejected by the scanner as a separate check.

7.6 Theory: why not just a regex for the program?

The language of token sequences is regular, but programming languages require balanced parentheses, matched braces, and nested structure. $\{(^n)^n\}$ is not regular, so after scanning we need a more powerful formalism: context-free grammars (next chapter).

7.7 Summary

- The scanner turns characters into (kind, lexeme) tokens using a DFA built from the regexes for each token kind (union, then subset construction).
- Decompositions are ambiguous; maximal munch takes the longest token each time.
- General maximal munch backs up to the last accepting state (worst case quadratic).

- Simplified maximal munch never backs up: linear time, but may reject decomposable inputs.
- Keywords: scan as identifiers then look them up; whitespace and comments are discarded.

Chapter 8

Context-Free Grammars

8.1 Motivation

Regular languages cannot describe nesting. Balanced parentheses, nested blocks and arithmetic expressions all require "memory" of unbounded depth. Context-free grammars (CFGs) add recursion, which is exactly enough to describe the syntactic structure of most programming languages.

8.2 Definitions

Definition 8.1. A **context-free grammar** is a 4-tuple $G = (N, T, P, S)$:

- N: a finite set of **non-terminals**;
- T: a finite set of **terminals** (the alphabet, disjoint from N);
- P: a finite set of **productions** $A \to \alpha$ with $A \in N$, $\alpha \in (N \cup T)^*$;
- $S \in N$: the **start symbol**.

Let $V = N \cup T$ be the vocabulary.

Conventions: a, b, c are terminals; A, B, C, S non-terminals; $w, x, y, z \in T^*$; $\alpha, \beta, \gamma \in V^*$. "Context-free" means a non-terminal A can be replaced by α regardless of what surrounds it.

Definition 8.2. $\alpha A \beta \Rightarrow \alpha\gamma\beta$ (*directly derives*) if $A \to \gamma \in P$. $\Rightarrow^*$ is the reflexive transitive closure. A **derivation** of w is a sequence $S \Rightarrow \alpha_1 \Rightarrow \cdots \Rightarrow w$. The language of G is $L(G) = \{w \in T^* : S \Rightarrow^* w\}$. A language is **context-free** if some CFG generates it.

Example 8.3. $S \to aSb \mid \varepsilon$ generates $\{a^n b^n : n \geq 0\}$ (not regular). Balanced parentheses: $S \to (S)S \mid \varepsilon$. Palindromes over $\{a, b\}$: $S \to aSa \mid bSb \mid a \mid b \mid \varepsilon$.

Every regular language is context-free: a DFA transition $\delta(q, a) = r$ becomes the production $Q \to aR$, and each accepting state Q gets $Q \to \varepsilon$.

8.3 Derivations and parse trees

Consider

$$E \to E + E \mid E * E \mid (E) \mid \texttt{id}.$$

A **leftmost derivation** always expands the leftmost non-terminal; a **rightmost derivation** the rightmost:

$$\text{leftmost: } E \Rightarrow E + E \Rightarrow \texttt{id} + E \Rightarrow \texttt{id} + E * E \Rightarrow \texttt{id} + \texttt{id} * E \Rightarrow \texttt{id} + \texttt{id} * \texttt{id}$$
$$\text{rightmost: } E \Rightarrow E + E \Rightarrow E + E * E \Rightarrow E + E * \texttt{id} \Rightarrow E + \texttt{id} * \texttt{id} \Rightarrow \texttt{id} + \texttt{id} * \texttt{id}$$

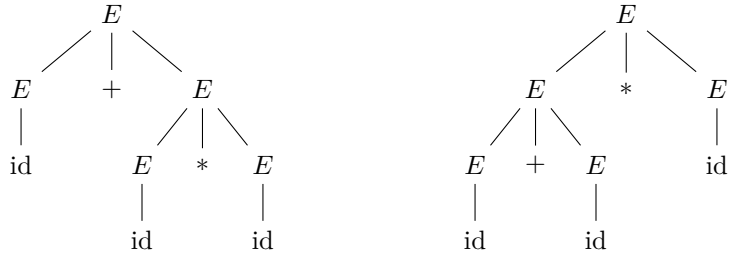
A **parse tree** records which production was applied to each non-terminal, forgetting the order. Root is S; each internal node A with children $X_1 \dots X_k$ corresponds to a production $A \to X_1 \dots X_k$; leaves read left-to-right give the derived string. Each parse tree corresponds to exactly one leftmost and exactly one rightmost derivation.

Top-down parsers (Chapter 9) construct leftmost derivations; bottom-up parsers (Chapter 10) construct rightmost derivations in reverse.

8.4 Ambiguity

Definition 8.4. A grammar is **ambiguous** if some word has two distinct parse trees (equivalently, two distinct leftmost derivations).

The expression grammar above is ambiguous: $\texttt{id} + \texttt{id} * \texttt{id}$ also has the leftmost derivation $E \Rightarrow E * E \Rightarrow E + E * E \Rightarrow \dots$



Left: $a + (b * c)$ (intended). Right: $(a + b) * c$.

Ambiguity matters because a compiler assigns *meaning* by walking the parse tree; two trees mean two different programs. Note: ambiguity is a property of the *grammar*. Some context-free languages are *inherently ambiguous* (every grammar for them is ambiguous), e.g. $\{a^i b^j c^k : i = j \text{ or } j = k\}$. Deciding whether a CFG is ambiguous is undecidable in general.

8.5 Precedence and associativity

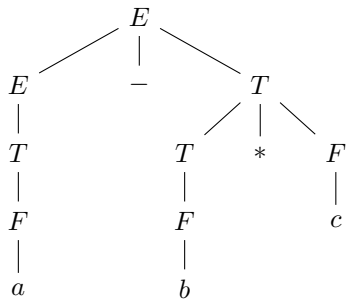
Two ways to handle ambiguity: (1) keep the ambiguous grammar and add disambiguation rules to the parser (as tools like yacc/bison do); (2) rewrite the grammar to be unambiguous. The standard rewrite introduces one non-terminal per precedence level:

$$E \to E + T \mid E - T \mid T$$
$$T \to T * F \mid T/F \mid F$$
$$F \to (E) \mid \texttt{id}$$

- **Precedence**: $*$ is generated lower in the tree (by T) than $+$ (by E), so it binds tighter. Operators lower in the tree are evaluated first.

- **Associativity**: $E \to E + T$ is *left-recursive*, so $a - b - c$ parses as $(a - b) - c$ (left-associative). Writing $E \to T + E$ would make it right-associative, which is wrong for $-$ and $/$ (but correct for e.g. exponentiation or assignment).

Example 8.5. Parse tree of $a - b * c$ in the unambiguous grammar:



The dangling else. $S \to$ `if` E S | `if` E S `else` S | ... is ambiguous for `if a if b x else y`. The convention is "else matches the nearest unmatched if". WLP4 sidesteps the issue by requiring braces and always having an else: `if (test) { stmts } else { stmts }`.

8.6 The WLP4 grammar (excerpt)

WLP4 ("Waterloo, Language Plus Pointers Plus Procedures") is the C-like language we compile. A fragment of its grammar:

```
start     -> BOF procedures EOF
procedures-> procedure procedures | main
procedure -> INT ID LPAREN params RPAREN LBRACE dcls statements RETURN expr SEMI RBRACE
main      -> INT WAIN LPAREN dcl COMMA dcl RPAREN LBRACE dcls statements RETURN expr SEMI
     RBRACE
dcl       -> type ID
type      -> INT | INT STAR
dcls      -> | dcls dcl BECOMES NUM SEMI | dcls dcl BECOMES NULL SEMI
statements-> | statements statement
statement -> lvalue BECOMES expr SEMI
           | IF LPAREN test RPAREN LBRACE statements RBRACE ELSE LBRACE statements RBRACE
           | WHILE LPAREN test RPAREN LBRACE statements RBRACE
           | PRINTLN LPAREN expr RPAREN SEMI
           | DELETE LBRACK RBRACK expr SEMI
test      -> expr EQ expr | expr NE expr | expr LT expr | ...
expr      -> term | expr PLUS term | expr MINUS term
term      -> factor | term STAR factor | term SLASH factor | term PCT factor
factor    -> ID | NUM | NULL | LPAREN expr RPAREN | AMP lvalue | STAR factor
           | NEW INT LBRACK expr RBRACK | ID LPAREN RPAREN | ID LPAREN arglist RPAREN
```

Note the `expr`/`term`/`factor` layering, exactly the precedence technique above. `BOF` and `EOF` are artificial tokens that make parsing algorithms cleaner.

8.7 Parsing as a problem

Parsing: given G and w, decide whether $w \in L(G)$ and, if so, produce a derivation (parse tree). General algorithms (CYK, Earley) run in $\mathcal{O}(n^3)$ time, too slow for compilers. Instead we restrict

to grammar classes that allow linear-time parsing with one token of lookahead: LL(1) (top-down) and LR(1)/LALR(1) (bottom-up).

8.8 Summary

- CFG = (N, T, P, S); derivations replace one non-terminal at a time.
- Parse trees abstract away derivation order; leftmost and rightmost derivations are canonical orders.
- Ambiguous grammar: some string has two parse trees, hence two meanings.
- Encode precedence with one non-terminal per level; left recursion gives left associativity.
- WLP4's grammar uses `expr`/`term`/`factor`; BOF/EOF delimit the input.

Chapter 9

Top-Down Parsing

9.1 Idea

A **top-down** parser builds the parse tree from the root down, producing a *leftmost derivation*. At every step it must decide which production to apply to the leftmost non-terminal. An LL(1) parser makes that decision by peeking at *one* token of input. ("LL(1)": read input **L**eft to right, produce a **L**eftmost derivation, **1** token of lookahead.)

We augment every grammar with a new start production $S' \to \vdash\ S\ \dashv$, where $\vdash$ (BOF) and $\dashv$ (EOF) are new terminals surrounding the input. Then there is always a lookahead symbol, even at the end.

9.2 Stack-based top-down parsing

The parser keeps a stack of grammar symbols representing the part of the derivation not yet matched. Invariant: *consumed input + stack (read top to bottom)* = current sentential form of the leftmost derivation.

```
push S';                          // or push the right-hand side of S' in reverse
for each token a in input (including BOF, EOF):
    while top of stack is a non-terminal A:
        if Predict[A][a] is empty: reject;
        pop A; push the RHS of Predict[A][a] in reverse order;
    if top of stack != a: reject;
    pop;                          // match
accept if stack is empty and input is exhausted;
```

Everything depends on the table Predict$[A][a]$: the production to use when A is on top of the stack and a is the next input token. A grammar is **LL(1)** iff every entry contains at most one production.

9.3 Nullable, FIRST and FOLLOW

Definition 9.1. For $\beta \in V^*$ and $A \in N$:

- Nullable(β) is true iff $\beta \Rightarrow^* \varepsilon$.
- First$(\beta) = \{a \in T : \beta \Rightarrow^* a\gamma\}$: terminals that can begin a string derived from β.

- Follow$(A) = \{b \in T : S' \Rightarrow^* \alpha A b \gamma\}$: terminals that can appear immediately after A in some sentential form.

Definition 9.2. Predict$(A, a) = \{A \to \beta : a \in \text{First}(\beta)\} \cup \{A \to \beta : \text{Nullable}(\beta) \text{ and } a \in \text{Follow}(A)\}$.

9.3.1 Computing them (fixed-point iteration)

All three are computed by iterating until nothing changes.

```
// Nullable
initialize Nullable[A] = false for all A
repeat
  for each A -> B1 ... Bk:
    if k == 0 or all Bi are nullable non-terminals: Nullable[A] = true
until no change

// First (for non-terminals)
initialize First[A] = {} for all A
repeat
  for each A -> B1 ... Bk:
    for i = 1..k:
      if Bi is a terminal: First[A] += {Bi}; break
      First[A] += First[Bi]
      if not Nullable[Bi]: break
until no change

// Follow
initialize Follow[A] = {} for all A
repeat
  for each A -> B1 ... Bk:
    for i = 1..k with Bi a non-terminal:
      Follow[Bi] += First(B(i+1) ... Bk)
      if Nullable(B(i+1) ... Bk): Follow[Bi] += Follow[A]
until no change
```

For a sequence: First$(X_1 \dots X_k) = \text{First}(X_1) \cup \text{First}(X_2)$ (if X_1 nullable) $\cup \dots$, stopping at the first non-nullable symbol; for a terminal a, First$(a) = \{a\}$.

9.4 A worked example

The unambiguous expression grammar from Chapter 8 is left-recursive and therefore not LL(1) (see below). Here is an equivalent LL(1) version:

$$
\begin{array}{ll}
0 & S' \to \vdash E \dashv \\
1 & E \to T E' \\
2 & E' \to + T E' \\
3 & E' \to \varepsilon \\
4 & T \to F T' \\
5 & T' \to * F T' \\
6 & T' \to \varepsilon \\
7 & F \to (E) \\
8 & F \to \texttt{id}
\end{array}
$$

	Nullable	First	Follow
E	no	$\{(, \texttt{id}\}$	$\{), \dashv\}$
E'	yes	$\{+\}$	$\{), \dashv\}$
T	no	$\{(, \texttt{id}\}$	$\{+,), \dashv\}$
T'	yes	$\{*\}$	$\{+,), \dashv\}$
F	no	$\{(, \texttt{id}\}$	$\{*, +,), \dashv\}$

Predict table (entries are rule numbers; blank = error):

	$\vdash$	$\dashv$	id	$+$	$*$	(	)
S'	0						
E			1			1	
E'		3		2			3
T			4			4	
T'		6		6	5		6
F			8			7	

No entry has two rules, so the grammar is LL(1).

Example 9.3 (Parsing $\vdash \texttt{id} + \texttt{id} * \texttt{id} \dashv$)**.** Stack shown with top on the left.

step	stack	remaining input	action
1	S'	$\vdash$ id + id * id $\dashv$	expand 0
2	$\vdash E \dashv$	$\vdash$ id + id * id $\dashv$	match $\vdash$
3	$E \dashv$	id + id * id $\dashv$	expand 1
4	$TE' \dashv$	id + id * id $\dashv$	expand 4
5	$FT'E' \dashv$	id + id * id $\dashv$	expand 8
6	id $T'E' \dashv$	id + id * id $\dashv$	match
7	$T'E' \dashv$	+ id * id $\dashv$	expand 6
8	$E' \dashv$	+ id * id $\dashv$	expand 2
9	$+TE' \dashv$	+ id * id $\dashv$	match
10	$TE' \dashv$	id * id $\dashv$	expand 4
11	$FT'E' \dashv$	id * id $\dashv$	expand 8
12	id $T'E' \dashv$	id * id $\dashv$	match
13	$T'E' \dashv$	* id $\dashv$	expand 5
14	$*FT'E' \dashv$	* id $\dashv$	match
15	$FT'E' \dashv$	id $\dashv$	expand 8
16	id $T'E' \dashv$	id $\dashv$	match
17	$T'E' \dashv$	$\dashv$	expand 6
18	$E' \dashv$	$\dashv$	expand 3
19	$\dashv$	$\dashv$	match
20	(empty)	(empty)	accept

The sequence of expansions (0,1,4,8,6,2,4,8,5,8,6,3) is exactly the leftmost derivation.

9.5 When a grammar is not LL(1)

9.5.1 Left recursion

If $A \to A\alpha \mid \beta$, then $\text{First}(\beta) \subseteq \text{First}(A\alpha)$, so both rules are predicted on the same token: a conflict. (A recursive-descent parser would loop forever.) **Fix**: rewrite as right recursion with a

new non-terminal:

$$A \to A\alpha \mid \beta \qquad \rightsquigarrow \qquad A \to \beta A', \quad A' \to \alpha A' \mid \varepsilon.$$

This is how $E \to E + T \mid T$ became $E \to TE'$, $E' \to +TE' \mid \varepsilon$.

Associativity is lost

The rewritten grammar's parse tree for $a - b - c$ hangs to the right. A compiler must re-associate when building the abstract syntax tree (or evaluate with an accumulator) to preserve left associativity. This is one reason LR parsing is preferred for expressions.

9.5.2 Common prefixes: left factoring

$A \to \alpha\beta_1 \mid \alpha\beta_2$ conflicts on First(α). Factor out the prefix: $A \to \alpha A'$, $A' \to \beta_1 \mid \beta_2$. E.g. WLP4's `factor -> ID | ID LPAREN RPAREN | ...` needs factoring for LL(1).

9.5.3 Limits

Some languages have no LL(1) grammar at all (e.g. $\{a^n b^m : n \geq m\}$ which needs unbounded lookahead to decide). LL(k) for larger k helps sometimes but tables grow quickly. Every LL(1) grammar is unambiguous.

9.6 Recursive descent

An LL(1) grammar can be parsed by hand by writing one function per non-terminal; the call stack plays the role of the parse stack.

```
Token la;                                   // lookahead
void match(Kind k) { if (la.kind != k) error(); la = next(); }
void E()  { T(); Eprime(); }
void Eprime() {
    if (la.kind == PLUS) { match(PLUS); T(); Eprime(); }
    else if (la.kind == RPAREN || la.kind == EOF_) { /* epsilon */ }
    else error();
}
void T()  { F(); Tprime(); }
void Tprime() {
    if (la.kind == STAR) { match(STAR); F(); Tprime(); }
    else if (la.kind == PLUS || la.kind == RPAREN || la.kind == EOF_) {}
    else error();
}
void F() {
    if (la.kind == ID) match(ID);
    else if (la.kind == LPAREN) { match(LPAREN); E(); match(RPAREN); }
    else error();
}
```

Each branch condition is exactly a row of the predict table. Many production compilers (GCC's and Clang's C++ front ends) use hand-written recursive descent.

9.7 Summary

- Top-down parsing = leftmost derivation; expand non-terminals on the stack, match terminals.
- Predict(A, a) uses First of the RHS, plus Follow(A) when the RHS is nullable.
- Compute Nullable, First, Follow by fixed-point iteration.
- LL(1) iff the predict table has no multiply-defined entries.
- Remove left recursion and left-factor to make a grammar LL(1); beware lost associativity.
- Recursive descent is a hand-coded LL(1) parser.

Chapter 10

Bottom-Up Parsing

10.1 Idea

A **bottom-up** parser builds the tree from the leaves up. It reads tokens onto a stack (**shift**) and, when the top of the stack matches the right-hand side of a production, replaces it with the left-hand side (**reduce**). Read in reverse, the reductions form a *rightmost derivation*. Such parsers are called **LR**: **L**eft-to-right scan, **R**ightmost derivation in reverse.

Invariant: *stack (bottom to top)* + *unread input* = a sentential form of the rightmost derivation.

Example 10.1. Grammar: $S' \to \vdash E \dashv$, $E \to E + T \mid T$, $T \to \texttt{id}$.

stack	input	action
	$\vdash$ id + id $\dashv$	shift
$\vdash$	id + id $\dashv$	shift
$\vdash$ id	+ id $\dashv$	reduce $T \to$ id
$\vdash T$	+ id $\dashv$	reduce $E \to T$
$\vdash E$	+ id $\dashv$	shift
$\vdash E+$	id $\dashv$	shift
$\vdash E+$ id	$\dashv$	reduce $T \to$ id
$\vdash E + T$	$\dashv$	reduce $E \to E + T$
$\vdash E$	$\dashv$	shift
$\vdash E \dashv$		reduce $S' \to \vdash E \dashv$, accept

The hard question is *when* to shift and when to reduce (and by which rule). Note that the stack in the third row could be reduced in many ways; we must only reduce when the result can still be extended to a valid parse.

10.2 Viable prefixes and items

Definition 10.2. A **viable prefix** is a prefix of a right sentential form that does not extend past the right end of the next handle to be reduced. The stack of a correct LR parser always holds a viable prefix.

Theorem 10.3 (Knuth, 1965)**.** *For any CFG, the set of viable prefixes is a* regular language.

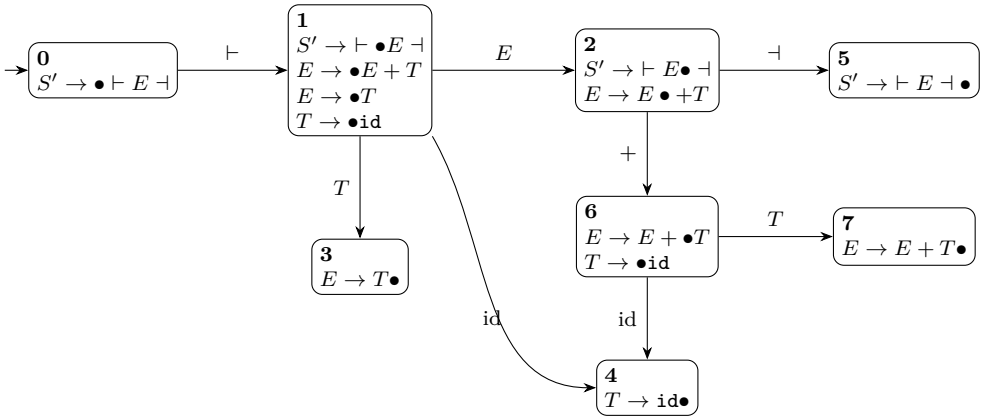
So a DFA can read the stack and tell us what to do. Its states are built from items.

Definition 10.4. An **LR(0) item** is a production with a dot somewhere in its right-hand side, e.g. $E \to E \bullet + T$. The dot marks how much of the RHS is already on the stack. An item with the dot at the end, $A \to \alpha\bullet$, is a **complete** (reduce) item.

10.3 Building the LR(0) automaton

1. Start state: the **closure** of $\{S' \to \bullet \vdash S \dashv\}$.
2. **Closure**: whenever a state contains $A \to \alpha \bullet B\beta$ with B a non-terminal, add $B \to \bullet\gamma$ for every production of B; repeat.
3. **Goto**: from state I on symbol X, take every item $A \to \alpha \bullet X\beta$ in I, move the dot over X to get $A \to \alpha X \bullet \beta$, and take the closure.
4. Repeat until no new states appear.

Example 10.5 (LR(0) automaton for $S' \to \vdash E \dashv$, $E \to E + T \mid T$, $T \to$ `id`)**.** States are boxes listing their items; edges are goto transitions.



States 3, 4, 5, 7 contain one complete item and nothing else: reduce states.

10.4 The LR parsing algorithm

Rather than re-running the DFA over the whole stack after every reduction, keep a stack of DFA *states* alongside the symbols.

```
stateStack.push(start);
for each token a (BOF, ..., EOF):
    // reduce as long as the current state says so
    while (Reduce[top(stateStack)][a] == A -> gamma) {
        output "A -> gamma";
        pop |gamma| symbols and states;
        push A; stateStack.push(Goto[top(stateStack)][A]);
    }
    if (Shift[top(stateStack)][a] is undefined) reject (error at token a);
    push a; stateStack.push(Shift[top(stateStack)][a]);
accept (after shifting EOF, reduce by S' -> BOF S EOF)
```

Each reduction costs $\mathcal{O}(|\gamma|)$ and every token is shifted once, so the parser runs in $\mathcal{O}(n)$ time for a grammar of fixed size. The output, a list of productions, is a reverse rightmost derivation; the CS 241 WLP4 parser prints it reversed (as a preorder traversal, i.e. a leftmost derivation) by building the tree on the fly.

Example 10.6 (Trace with states)**.** Parsing ⊢ `id` + `id` ⊣ with the automaton above:

state stack	input	action
0	$\vdash$ id + id $\dashv$	shift $\to$ 1
0 1	id + id $\dashv$	shift $\to$ 4
0 1 4	+ id $\dashv$	reduce $T \to$id; Goto(1,T)=3
0 1 3	+ id $\dashv$	reduce $E \to T$; Goto(1,E)=2
0 1 2	+ id $\dashv$	shift $\to$ 6
0 1 2 6	id $\dashv$	shift $\to$ 4
0 1 2 6 4	$\dashv$	reduce $T \to$id; Goto(6,T)=7
0 1 2 6 7	$\dashv$	reduce $E \to E + T$ (pop 3); Goto(1,E)=2
0 1 2	$\dashv$	shift $\to$ 5; accept

10.5 Conflicts

A state is problematic if it has

- a **shift-reduce conflict**: a complete item $A \to \alpha\bullet$ and an item $B \to \beta \bullet a\gamma$ (could shift a or reduce), or
- a **reduce-reduce conflict**: two complete items $A \to \alpha\bullet$ and $B \to \beta\bullet$.

A grammar is **LR(0)** iff its LR(0) automaton has no conflicts: then every reduce state can reduce without looking at the next token.

Example 10.7 (LR(0) fails)**.** Make + right-associative: $E \to T + E \mid T$, $T \to$ `id`. After reading T the state contains

$$E \to T \bullet + E \qquad \text{and} \qquad E \to T\bullet$$

a shift-reduce conflict: should we reduce T to E, or shift the +?

10.6 SLR(1)

Simple LR(1) uses the same LR(0) automaton, but only reduces by $A \to \alpha\bullet$ when the lookahead is in Follow(A).

In the example above, Follow(E) = $\{\dashv\}$, so in that state: on + shift, on $\dashv$ reduce $E \to T$. The conflict is gone. A grammar is SLR(1) iff for every state: the Follow sets of its complete items are pairwise disjoint and disjoint from the terminals that can be shifted.

10.7 LR(1) and LALR(1)

Follow sets are global: Follow(A) includes everything that can follow A *anywhere*, which may be too coarse in a particular state. LR(1) refines the items.

Definition 10.8. An **LR(1) item** $[A \to \alpha\bullet\beta,\ a]$ pairs an LR(0) item with a lookahead terminal a, meaning "we are in the middle of $A \to \alpha\beta$, and after it the input should continue with a". Closure: for $[A \to \alpha \bullet B\beta,\ a]$ add $[B \to \bullet\gamma,\ b]$ for every $b \in$ First(βa). Reduce by $[A \to \alpha\bullet,\ a]$ only when the lookahead is a.

LR(1) is very powerful: almost all programming-language grammars are LR(1), and LR(1) grammars describe exactly the deterministic context-free languages. But LR(1) automata can have many more states than LR(0) (several copies of a state differing only in lookaheads).

LALR(1) (look-ahead LR) merges LR(1) states with identical *cores* (the same LR(0) items), unioning their lookaheads. The result has the same number of states as the LR(0) automaton but more precise lookaheads than SLR. Merging cannot introduce new shift-reduce conflicts but can introduce reduce-reduce conflicts.

Example 10.9 (LR(1) but not LALR(1))**.** $S \to aAd \mid bBd \mid aBe \mid bAe$, $A \to c$, $B \to c$. After $a\,c$ the LR(1) state is $\{[A \to c\bullet, d], [B \to c\bullet, e]\}$; after $b\,c$ it is $\{[A \to c\bullet, e], [B \to c\bullet, d]\}$. Both have the same core, so LALR merges them into $\{[A \to c\bullet, d/e], [B \to c\bullet, d/e]\}$: a reduce-reduce conflict.

method	automaton	reduce $A \to \alpha\bullet$ when lookahead is in
LR(0)	LR(0) items	(always)
SLR(1)	LR(0) items	Follow(A)
LALR(1)	LR(0) cores, merged LR(1) lookaheads	state-specific lookahead set
LR(1)	LR(1) items	item's own lookahead

Strength: LR(0) $\subset$ SLR(1) $\subset$ LALR(1) $\subset$ LR(1) (as grammar classes). Tools such as yacc and bison generate LALR(1) parsers. Every LL(1) grammar is LR(1), but not vice versa; in particular LR parsers handle left recursion naturally (and in fact prefer it, since it keeps the stack shallow).

10.8 Parsing WLP4

In the assignments, the WLP4 grammar's LR(1) DFA (transitions and reductions) was provided in a file; we implemented the driver loop above and output the parse tree. Error reporting: when no shift is possible, report the position of the offending token.

10.9 Summary

- Shift tokens; reduce when the stack top is a handle. Reductions give a rightmost derivation in reverse.
- The viable prefixes form a regular language, recognised by the LR(0) item automaton (closure + goto).
- Keep a stack of states so each step is $\mathcal{O}(1)$ amortised; overall linear time.
- Conflicts: shift-reduce and reduce-reduce. LR(0) allows none.
- SLR(1) uses Follow sets; LR(1) attaches lookaheads to items; LALR(1) merges LR(1) states with equal cores (may create reduce-reduce conflicts).

Chapter 11

Context-Sensitive Analysis

11.1 Beyond context-free

Many rules of a programming language cannot be expressed by a CFG (or would make it hopelessly complicated):

- a variable must be declared before use, and not declared twice in the same scope;
- a procedure must be called with the right number and types of arguments;
- operands must have compatible types.

The language $\{wcw : w \in \{a, b\}^*\}$ ("declare w, then use w") is not context-free. So after parsing, the compiler performs **context-sensitive (semantic) analysis** by traversing the parse tree with auxiliary data structures. A program that passes all checks is a valid WLP4 program; only valid programs are given to the code generator.

11.2 The WLP4 language

WLP4 is a tiny subset of C++:

- Two types: `int` (32-bit signed) and `int*` (pointer to int).
- Procedures return `int`; the last procedure is `wain`, which takes two parameters (the second must be `int`) and is the entry point.
- All local declarations appear at the top of a procedure, before any statements, and must be initialised: `int x = 5;` or `int* p = NULL;`.
- Statements: assignment; `if` with a mandatory `else` (braces required around both branches); `while`; printing with `println`; and freeing with `delete []`.
- Expressions: `+ - * / %`, comparisons `== != < <= > >=` (only in tests), address-of `&`, dereference `*`, `new int[n]`, `NULL`, and procedure calls.
- No globals, no nested procedures; a procedure must be declared before it is called.

```
int sum(int* a, int n) {
    int i = 0;
    int s = 0;
    while (i < n) {
        s = s + *(a + i);
        i = i + 1;
    }
    return s;
}
int wain(int* a, int n) {
    return sum(a, n);
}
```

11.3 Symbol tables

Definition 11.1. A **symbol table** maps identifiers to their attributes: for variables, the type (and later the stack offset); for procedures, the signature (parameter types).

WLP4 has two levels of scope:

- a **global** table of procedures: name $\mapsto$ (signature, local table);
- one **local** table per procedure: parameter and variable names $\mapsto$ type.

A variable and a procedure may share a name; inside a procedure a local variable shadows a procedure name for the purpose of calls (`f()` where `f` is a local variable is an error).

Building it. Traverse the parse tree:

- At `procedure`: error if the name is already in the global table; create a new local table; collect the parameter types into the signature.
- At each `dcl`: error if the name is already in the current local table; otherwise insert it with its type (`type -> INT` gives `int`, `type -> INT STAR` gives `int*`).
- At each use (`factor -> ID`, `lvalue -> ID`): error if not in the current local table.
- At each call `ID(...)`: error if the name is a local variable or not a previously declared procedure (recursion is allowed, so the current procedure is in the table as soon as its signature is read).

```
struct ProcInfo { std::vector<Type> params; std::map<std::string, Type> locals; };
std::map<std::string, ProcInfo> procs;

void collectDcl(Tree *dcl, ProcInfo &p) {
    std::string name = dcl->children[1]->lexeme;
    Type t = (dcl->children[0]->children.size() == 2) ? INTSTAR : INT;
    if (p.locals.count(name)) throw SemanticError("duplicate " + name);
    p.locals[name] = t;
}
```

11.4 Type checking

We compute a type for each expression bottom-up (a post-order traversal), and check that statements are **well-typed**. The rules can be written as inference rules; here as a table.

11.4.1 Expressions

expression	requirement	type
`NUM`		`int`
`NULL`		`int*`
`ID`	declared	declared type
`(E)`		type of E
`&E`	E an lvalue of type `int`	`int*`
`*E`	E : `int*`	`int`
`new int[E]`	E : `int`	`int*`
`E1 * E2`, `/`, `%`	both `int`	`int`
`E1 + E2`	`int + int`	`int`
	`int* + int` or `int + int*`	`int*`
`E1 - E2`	`int - int`	`int`
	`int* - int`	`int*`
	`int* - int*`	`int`
`f(E1,...,En)`	arity and each type match the signature	`int`

The pointer rules mirror C: adding an integer to a pointer moves it by that many *elements*; subtracting two pointers gives the distance in elements. `int* + int*` is an error.

11.4.2 Statements and tests

- `lvalue = expr;` : both sides have the same type.
- `println(E);` : E is `int`.
- `delete [] E;` : E is `int*`.
- tests `E1 op E2` : E_1 and E_2 have the same type (comparing pointers is allowed, comparing a pointer to an int is not).
- `if`/`while`: the test is well-typed and the bodies are well-typed.

11.4.3 Declarations and procedures

- `int x = NUM;` requires `int`; `int* p = NULL;` requires `int*`.
- The `return` expression of every procedure (including `wain`) must be `int`.
- The second parameter of `wain` must be `int`.

```
Type typeOf(Tree *t) {
    const std::string &r = t->rule;
    if (r == "factor NUM")  return INT;
    if (r == "factor NULL") return INTSTAR;
    if (r == "factor ID")   return lookup(t->children[0]->lexeme);
    if (r == "factor STAR factor") {
```

```
        if (typeOf(t->children[1]) != INTSTAR) throw SemanticError("deref non-pointer");
        return INT;
    }
    if (r == "expr expr PLUS term") {
        Type a = typeOf(t->children[0]), b = typeOf(t->children[2]);
        if (a == INT && b == INT) return INT;
        if (a != b) return INTSTAR;        // exactly one pointer
        throw SemanticError("int* + int*");
    }
    // ... remaining rules ...
}
```

It is convenient to store the computed type in each tree node, because the code generator needs it later (e.g. to scale pointer arithmetic by 4).

11.5 Attribute grammars (brief)

More generally, semantic analysis can be described by an *attribute grammar*: each grammar symbol carries attributes (like *type*) and each production has rules for computing them. *Synthesized* attributes flow up the tree (types of expressions); *inherited* attributes flow down (the current symbol table). WLP4 type checking uses mostly synthesized attributes plus a symbol table passed down.

11.6 Errors and pitfalls

- Check both "declared twice" and "used but not declared", and do it per procedure.
- Recursion: insert the procedure into the global table before checking its body.
- A call to an undeclared (or later-declared) procedure is an error: WLP4 requires declaration before use.
- Remember `&` needs an lvalue: `&(x)` is fine (parenthesised lvalue), `&5` is not parseable anyway.
- Do not stop at the first type of each subtree: the whole tree must be checked, even statements whose value is never used.

11.7 Summary

- Context-sensitive rules (declarations, types, arity) are checked by traversing the parse tree.
- Symbol tables: one global table of procedure signatures and one local table per procedure.
- WLP4 types are `int` and `int*`; pointer arithmetic follows C rules.
- Compute expression types bottom-up; check statements, tests, declarations and returns.
- Annotate the tree with types; the code generator will use them.

Chapter 12

Code Generation

12.1 Overview

The code generator traverses the annotated parse tree of a valid WLP4 program and emits MIPS assembly. There are infinitely many correct translations; our goal is a *simple, correct* one, structured so each construct's code can be generated independently (syntax-directed translation). We write code(X) for the assembly produced for subtree X.

Invariant

code(*expr*) leaves the value of *expr* in `$3`, and leaves every other register (except scratch `$5`–`$7`) and the stack as they were.

12.2 Conventions

register	use
`$0`	zero
`$1`, `$2`	parameters of `wain` (and arguments to runtime routines)
`$3`	result of every expression and procedure
`$4`	constant 4
`$5`–`$7`	scratch
`$11`	constant 1
`$29`	frame pointer
`$30`	stack pointer
`$31`	return address

Helper "macros" the generator emits often:

```
; push($r)
sw $r, -4($30)
sub $30, $30, $4
; pop($r)
add $30, $30, $4
lw $r, -4($30)
```

12.3 Variables and the stack frame

All variables live on the stack, addressed relative to a **frame pointer** `$29` that is fixed for the whole procedure (unlike `$30`, which moves as temporaries are pushed). The symbol table records each variable's offset.

Prologue of wain. Import the runtime, set up constants, and push the parameters and locals:

```
.import print
.import init
.import new
.import delete
lis $4
.word 4
lis $11
.word 1
sub $29, $30, $4     ; $29 points at first variable slot
sw $1, -4($30)       ; a at offset 0
sub $30, $30, $4
sw $2, -4($30)       ; b at offset -4
sub $30, $30, $4
; locals: int x = 7;  -> offset -8
lis $5
.word 7
sw $5, -4($30)
sub $30, $30, $4
```

The k-th variable (counting from 0) is at offset $-4k$ from `$29`. Reading a variable: `lw $3, off($29)`; writing: `sw $3, off($29)`.

Epilogue. `add $30, $29, $4` (pop the frame) then `jr $31`.

12.4 Expressions

12.4.1 Constants and variables

```
; code(NUM)              ; code(ID)
lis $3                   lw $3, off($29)
.word NUM
```

(`NULL` is represented by the value 1, not 0: it is not word-aligned, so dereferencing NULL crashes the emulator, which is the desired behaviour. Use `add $3, $0, $11`.)

12.4.2 Binary operators

Use the stack to hold the left operand while computing the right one:

```
; code(e1 + e2), both int
code(e1)
push($3)
code(e2)
pop($5)
add $3, $5, $3
```

Similarly `sub`; for `*` use `mult $5, $3; mflo $3`; for `/` use `div $5, $3; mflo $3`; for `%` use `mfhi $3`. This recursive scheme works for arbitrarily nested expressions because each subexpression restores the stack.

> An easy optimisation: if `e2` is a constant or variable, compute it straight into a register without the push/pop. Another: allocate a different temporary register per nesting level and only spill to the stack when they run out.

12.4.3 Pointer arithmetic

Types (from semantic analysis) decide the code:

expression	code after $\$5 = e_1$, $\$3 = e_2$
`int* + int`	`mult $3, $4; mflo $3; add $3, $5, $3`
`int + int*`	`mult $5, $4; mflo $5; add $3, $5, $3`
`int* - int`	`mult $3, $4; mflo $3; sub $3, $5, $3`
`int* - int*`	`sub $3, $5, $3; div $3, $4; mflo $3`

12.4.4 Pointers: `*`, `&`, and assignment through pointers

```
; code(*e)          ; code(&ID)              ; code(&(*e)) = code(e)
code(e)             lis $3
lw $3, 0($3)        .word off
                    add $3, $3, $29
```

Assignment:

```
; ID = e;           ; *e1 = e2;
code(e)             code(e1)          ; address
sw $3, off($29)     push($3)
                    code(e2)          ; value
                    pop($5)
                    sw $3, 0($5)
```

12.5 Tests

Produce 1 (true) or 0 (false) in `$3`. Use `slt` for `int` and `sltu` for pointers (addresses are unsigned).

test	code after $\$5 = e_1$, $\$3 = e_2$
$e_1 < e_2$	`slt $3, $5, $3`
$e_1 > e_2$	`slt $3, $3, $5`
$e_1 \neq e_2$	`slt $6, $3, $5; slt $7, $5, $3; add $3, $6, $7`
$e_1 = e_2$	($\neq$ code), then `sub $3, $11, $3`
$e_1 \geq e_2$	`slt $3, $5, $3; sub $3, $11, $3`
$e_1 \leq e_2$	`slt $3, $3, $5; sub $3, $11, $3`

12.6 Control flow

Each `if` and `while` needs unique labels; keep a global counter.

```
; if (T) {S1} else {S2}
code(T)
beq $3, $0, else4
code(S1)
beq $0, $0, endif4
else4:
code(S2)
endif4:
```

```
; while (T) {S}
loop7:
code(T)
beq $3, $0, endWhile7
code(S)
beq $0, $0, loop7
endWhile7:
```

A sequence of statements is just the concatenation of their code.

12.6.1 Printing

`print` (from `print.merl`) prints `$1` as a decimal number and a newline. Since `$1` holds `wain`'s parameter we save it:

```
; println(e);
push($1)
code(e)
add $1, $3, $0
push($31)
lis $5
.word print
jalr $5
pop($31)
pop($1)
```

12.7 Procedures

Naming. Prefix every WLP4 procedure label, e.g. `Ffoo`, so it cannot clash with generated labels (`else4`) or runtime routines (`print`, `new`). `wain`'s code comes first in the output so execution starts there; other procedures follow.

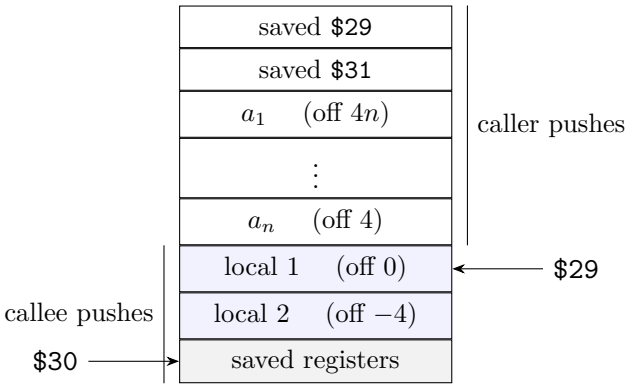
Caller. Saves `$29` and `$31`, pushes arguments in order, calls, pops the arguments, restores.

```
; code(f(e1, ..., en))
push($29)
push($31)
code(e1)
push($3)
...
code(en)
push($3)
lis $5
.word Ff
jalr $5
; pop n arguments (n adds, or one add of 4n)
pop($31)
pop($29)
```

Callee. Sets up its frame, runs its body, and tears the frame down:

```
Ff:
sub $29, $30, $4     ; new frame pointer
; push locals (initial values)
; push registers this procedure will modify ($5, $6, $7, ...)
code(statements)
code(return expr)    ; result in $3
; pop saved registers
add $30, $29, $4     ; discard locals
jr $31
```

Offsets. With n parameters, the caller pushed $a_1, \ldots, a_n$, so after `sub $29, $30, $4`, parameter a_i is at offset $4(n - i + 1)$ (positive) from `$29`: a_n at 4, a_1 at $4n$. Locals are at $0, -4, -8, \ldots$ as in `wain`. The symbol table is filled with these offsets while processing the procedure's declarations.



Who saves what

Here the caller saves `$29` and `$31`; the callee saves any other register it modifies. `$4`, `$11` are never modified. Mixing up the order of pushes and pops is the most common bug: draw the stack!

12.8 Dynamic memory: `new` and `delete`

The runtime `alloc.merl` provides `init`, `new` and `delete`.

- `init` must be called once in `wain`'s prologue (with `$31` saved). It expects `$1`/`$2` as given by `mips.array`; if `wain`'s first parameter is `int` (`mips.twoints`), set `$2` to 0 first.
- `new`: number of words in `$1`; returns address in `$3`, or 0 on failure. WLP4 must return NULL on failure, so convert 0 to 1.
- `delete`: address in `$1`. Skip the call when the pointer is NULL.

```
; code(new int[e])
code(e)
add $1, $3, $0
push($31)
lis $5
```

```
.word new
jalr $5
pop($31)
bne $3, $0, 1       ; success: skip next instruction
add $3, $11, $0     ; failure: return NULL (1)
```

```
; code(delete [] e;)
code(e)
beq $3, $11, skipDelete9   ; never free NULL
add $1, $3, $0
push($31)
lis $5
.word delete
jalr $5
pop($31)
skipDelete9:
```

(Also save `$1` around these if it is still needed.)

12.9 Putting it together

The output is assembled to MERL and linked with the runtime:

```
wlp4scan < prog.wlp4 | wlp4parse | wlp4gen > prog.asm
cs241.linkasm < prog.asm > prog.merl
cs241.linker prog.merl print.merl alloc.merl > exec.mips
mips.twoints exec.mips      # or mips.array
```

Example 12.1 (Complete output for a small program)**.** Imports and the call to `init` are omitted since this program uses no runtime routines.

```
int wain(int a, int b) { return a + b * 2; }
```

```
lis $4
.word 4
lis $11
.word 1
sub $29, $30, $4
sw $1, -4($30)
sub $30, $30, $4     ; a at 0
sw $2, -4($30)
sub $30, $30, $4     ; b at -4
lw $3, 0($29)        ; a
sw $3, -4($30)
sub $30, $30, $4     ; push
lw $3, -4($29)       ; b
sw $3, -4($30)
sub $30, $30, $4     ; push
lis $3
.word 2
add $30, $30, $4
lw $5, -4($30)       ; pop b
mult $5, $3
```

```
mflo $3              ; b*2
add $30, $30, $4
lw $5, -4($30)       ; pop a
add $3, $5, $3       ; a + b*2
add $30, $29, $4
jr $31
```

12.10 Summary

- Syntax-directed translation: each construct's code is built from its children's code.
- Every expression leaves its result in `$3`; temporaries go on the stack.
- Variables are at fixed offsets from the frame pointer `$29`.
- Types select instructions: scale by 4 for pointer arithmetic, `sltu` for pointer comparisons.
- Unique labels for `if`/`while`; prefix procedure names.
- Calls: caller saves `$29`, `$31` and pushes arguments; callee sets `$29`, pushes locals and saved registers.
- `new`/`delete`/`print` come from runtime MERL modules linked in; NULL is 1.

Chapter 13

Optimization and Memory Management

13.1 Optimization

The code generator of Chapter 12 is correct but wasteful: it pushes and pops for every binary operator and reloads variables constantly. **Optimization** transforms a program into an equivalent one that is faster or smaller. ("Optimization" is a misnomer: finding truly optimal code is undecidable, so we only *improve*.) In CS 241 the final assignment rewarded shorter generated code, measured in instructions.

Correctness first

An optimization must preserve the observable behaviour of every valid program, including output, return value and (for WLP4) memory effects. Most optimizations need some proof that a condition holds (e.g. "this variable is not modified in between").

13.1.1 Constant folding and propagation

Constant folding: evaluate expressions whose operands are compile-time constants. `x = 2 + 3 * 4;` becomes `x = 14;`, saving the whole push/pop sequence:

```
lis $3
.word 14
```

Constant propagation: if `int x = 5;` and `x` is never assigned, every use of `x` can be replaced by 5, which may enable more folding. Folding must respect the target's semantics (32-bit wraparound, truncating division, and never fold a division by zero into a compile-time crash).

13.1.2 Common subexpression elimination

In `a = (b+c)*(b+c);` compute `b+c` once and reuse it, provided neither `b` nor `c` changes in between. Pointer stores complicate this (*aliasing*): `*p = 0` might modify `b` if `p == &b`.

13.1.3 Dead code elimination

Remove code whose result is never used or that can never execute, e.g. the body of `while (1 > 2) {...}` or an assignment to a variable that is overwritten before being read. Careful: `println`

and procedure calls have side effects and cannot be removed.

13.1.4 Strength reduction and peephole optimization

Replace expensive operations by cheaper ones: multiplying by 2 becomes `add $3, $3, $3`. A **peephole optimizer** scans a small window of emitted instructions for patterns:

```
; push($3) immediately followed by pop($5)
sw $3, -4($30)
sub $30, $30, $4
add $30, $30, $4
lw $5, -4($30)
; ==> equivalent to
add $5, $3, $0
```

13.1.5 Register allocation

Loads and stores dominate naive code. **Register allocation** keeps variables and temporaries in registers instead of on the stack. Since MIPS has 20 free registers, we need to share them among variables whose lifetimes do not overlap.

- A variable is **live** at a point if its current value may be read later.
- Two variables **interfere** if they are live at the same time.
- Build the **interference graph** (one vertex per variable, edge when they interfere). Assigning registers is **graph colouring** with k colours $= k$ registers.
- Graph colouring is NP-hard, so compilers use heuristics (e.g. Chaitin's: repeatedly remove a vertex with degree $< k$; if none exists, choose one to *spill* to memory).

Example 13.1. Live ranges in a straight-line fragment:

```
int a = 1;      // a live
int b = a + 2;  // a, b live
int c = b * b;  // b, c live (a dead if not used later)
return c + 1;
```

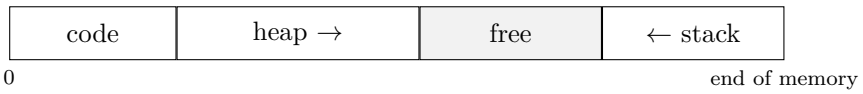
Interference edges: a–b, b–c. Two registers suffice: $a, c \mapsto$ `$5`, $b \mapsto$ `$6`.

13.1.6 Procedure-level optimizations

- **Inlining**: replace a call by the body of the callee, saving call overhead and enabling further optimization (at the cost of code size).
- **Tail-call optimization**: if the last action of a procedure is `return f(...)`, reuse the current frame and jump instead of calling. Tail recursion then runs in constant stack space (like a loop).
- Only save registers that are actually modified.

13.2 Memory management

WLP4's `new` and `delete` need a **heap**: a region of memory (between the program and the stack) from which blocks are allocated in arbitrary order and freed in arbitrary order (unlike the LIFO stack).



13.2.1 Explicit allocation with a free list

The allocator keeps a **free list**: a linked list of free blocks, stored *in* the free blocks themselves (each holds its size and a pointer to the next free block). Each allocated block has a hidden header word storing its size so `delete` knows how much to free; `new` returns the address just after the header.

Allocation of n words: search the free list for a block of size $\geq n+1$; split it, returning one part and leaving the remainder on the list. Strategies:

- **First fit**: the first block that is large enough (fast).
- **Best fit**: the smallest block that is large enough (less waste per allocation, but leaves tiny useless fragments; requires a full scan).
- **Worst fit**: the largest block (leaves big remainders).

Deallocation : insert the block back into the free list (often kept sorted by address) and **coalesce** it with adjacent free neighbours to form larger blocks.

Definition 13.2. Fragmentation: free memory is broken into many small non-contiguous blocks, so a request can fail even though the total free space would suffice. *External* fragmentation is wasted space between blocks; *internal* fragmentation is wasted space inside a block that is bigger than requested.

Example 13.3. Heap of 100 words. Allocate $A = 20$, $B = 30$, $C = 20$, then free B. Free space: 30 (where B was) + 30 at the end = 60 words, but a request for 40 words fails: external fragmentation.

No allocator can avoid fragmentation for all request sequences; without moving blocks (compaction) some sequences force heap usage much larger than the live data.

13.2.2 The buddy system

Restrict block sizes to powers of two. The heap starts as one block of size 2^k.

- To allocate n words, round up to the smallest power $2^j \geq n+1$. Take a free block of size 2^j; if none, split a larger block in half repeatedly. The two halves of a split are **buddies**.
- To free a block, check whether its buddy is also free; if so, merge them into a block twice the size, and repeat upward.

A block of size 2^j at offset a (relative to the heap start) has its buddy at offset $a \oplus 2^j$ (flip bit j), so finding buddies is cheap. Keep one free list per size. Trade-off: fast allocation and coalescing, but internal fragmentation up to about 50%.

Example 13.4. Heap 64 words. Request 7 words (+1 header = 8): split $64 \to 32+32 \to 16+16 \to 8+8$; return the first 8. Request 14 words ($\to$ 16): use the free 16. Freeing the 8 merges it with its free buddy 8 to give 16; that 16's buddy is allocated, so merging stops.

13.3 Garbage collection

In languages like Racket, Java or Python, the programmer never calls `delete`. The runtime automatically reclaims **garbage**: heap blocks that can no longer be reached by the program.

Definition 13.5. The **roots** are pointers held outside the heap (registers, stack, globals). A heap block is **reachable** if it can be reached from a root by following pointers. Unreachable blocks are garbage.

13.3.1 Reference counting

Each block stores the number of pointers to it. Assigning a pointer increments the new target's count and decrements the old target's; when a count reaches 0 the block is freed (recursively decrementing what it points to).

+ Garbage reclaimed immediately and incrementally; no long pauses.
− Every pointer assignment costs extra work; counts take space.
− **Cycles** are never reclaimed: two blocks pointing to each other keep each other's count at 1 even when unreachable.

13.3.2 Mark and sweep

1. **Mark**: starting from the roots, traverse (DFS) all reachable blocks and set a mark bit in each.
2. **Sweep**: scan the entire heap; unmarked blocks are added to the free list; clear marks on the others.

Handles cycles. Cost: marking is proportional to live data, sweeping to the whole heap. The program is paused while collecting ("stop the world"), and the heap remains fragmented.

13.3.3 Copying collection

Split the heap into two halves, *from-space* and *to-space*; allocate only in from-space by bumping a pointer (very fast). When it fills:

1. Copy every reachable block from from-space into to-space, contiguously, leaving a *forwarding address* in the old copy so each block is copied once and pointers to it can be updated (Cheney's algorithm does this with a BFS using to-space itself as the queue).
2. Swap the roles of the two halves.

+ Time proportional to live data only; compacts the heap (no fragmentation); allocation is a pointer bump.

− Only half the memory is usable at a time; objects move, so all pointers must be known precisely.

Modern collectors are *generational*: most objects die young, so new objects are allocated in a small nursery that is collected often (by copying), and survivors are promoted to an older generation collected rarely (often by mark-sweep-compact). Many also run incrementally or concurrently to avoid long pauses.

	reclaims cycles	pause	fragmentation
reference counting	no	short, incremental	yes
mark and sweep	yes	proportional to heap	yes
copying	yes	proportional to live data	no (compacts)

13.4 Summary

- Optimizations: constant folding/propagation, CSE, dead code, strength reduction, peephole, inlining, tail calls; all must preserve behaviour.
- Register allocation = colouring the interference graph of live ranges; spill when out of registers.
- Heaps use free lists (first/best/worst fit) with headers and coalescing; fragmentation is unavoidable in general.
- Buddy system: power-of-two blocks, buddy at $a \oplus 2^j$, fast merges, internal fragmentation.
- GC: reference counting (no cycles), mark and sweep (whole-heap sweep), copying (compacting, half the memory).